

Chapter 6

The Transport Layer

传输层



The Transport Layer

- The task of the transport layer:
 - To provide efficient, reliable, cost-effective data transport from source machine to the destination machine
- 传输层的任务：
 - 为源端到目的端提供可靠、价格合理的数据传输

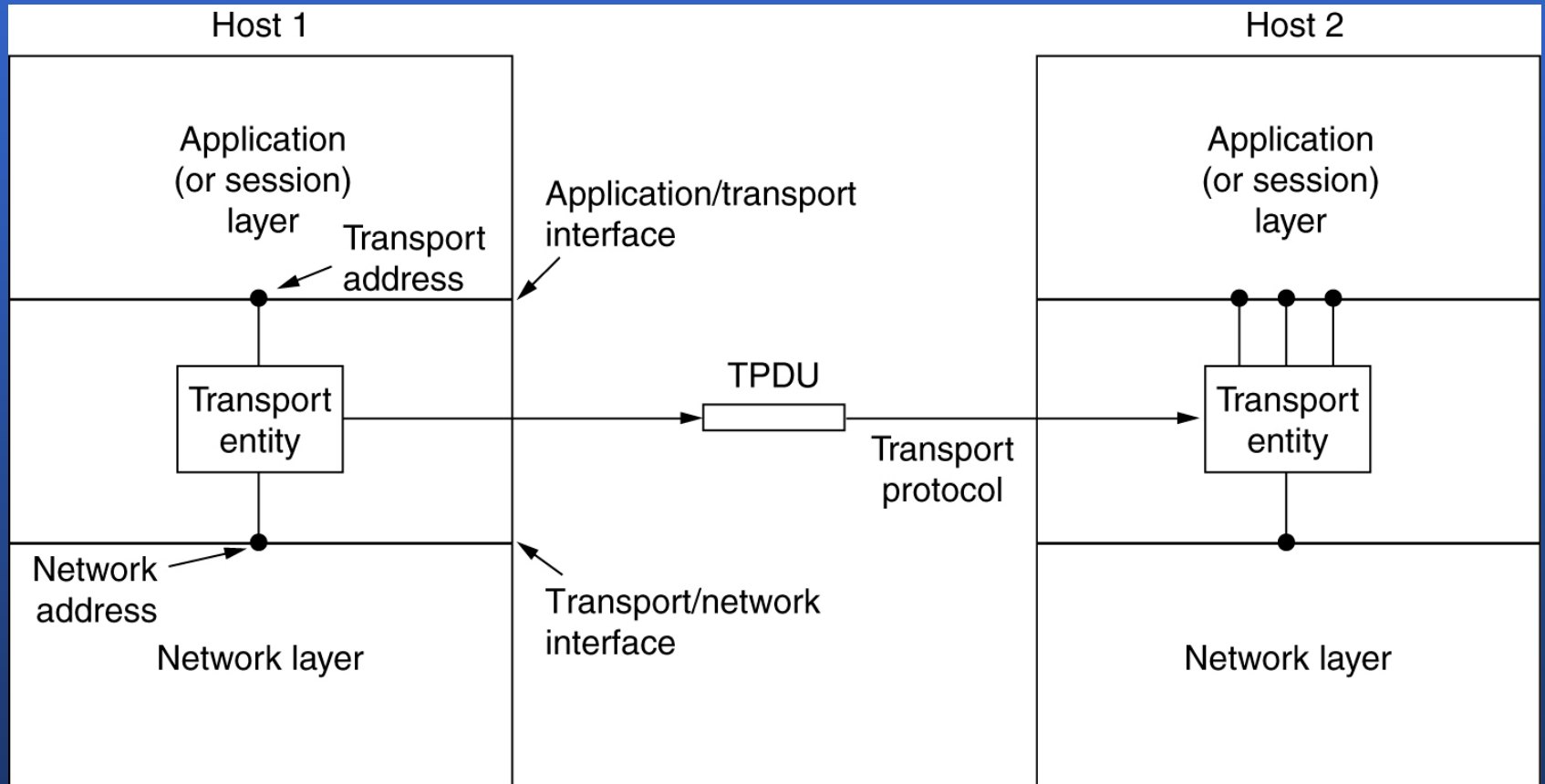


6.1 The Transport Service

- Services Provided to the Upper Layers
- Transport Service Primitives
- Berkeley Sockets
- An Example of Socket Programming:
 - An Internet File Server



6.1.1 Services Provided to the Upper Layers



The network, transport, and application layers.

Services Provided to the Upper Layers(2)

- 最终目标
 - 为用户（应用进程）提供有效、可靠、价格合理的服务
 - 网络层、传输层和应用层
- 传输服务类型：
 - 面向连接， 三个阶段：建立连接、数据传输和释放连接
 - 无连接
- OSI模型划分为2大部分：
 - 1-4层：传输服务提供者(Transport service provider)
 - 5-7层：传输服务用户(Transport service user)



6.1.2 Transport Service Primitives

- 传输服务与网络服务的区别
 - 网络服务不太可靠，而面向连接的传输服务则是可靠的
 - 服务对象不同
 - 网络层的数据交换比传输层复杂的多
- 一个简单传输服务的原语：

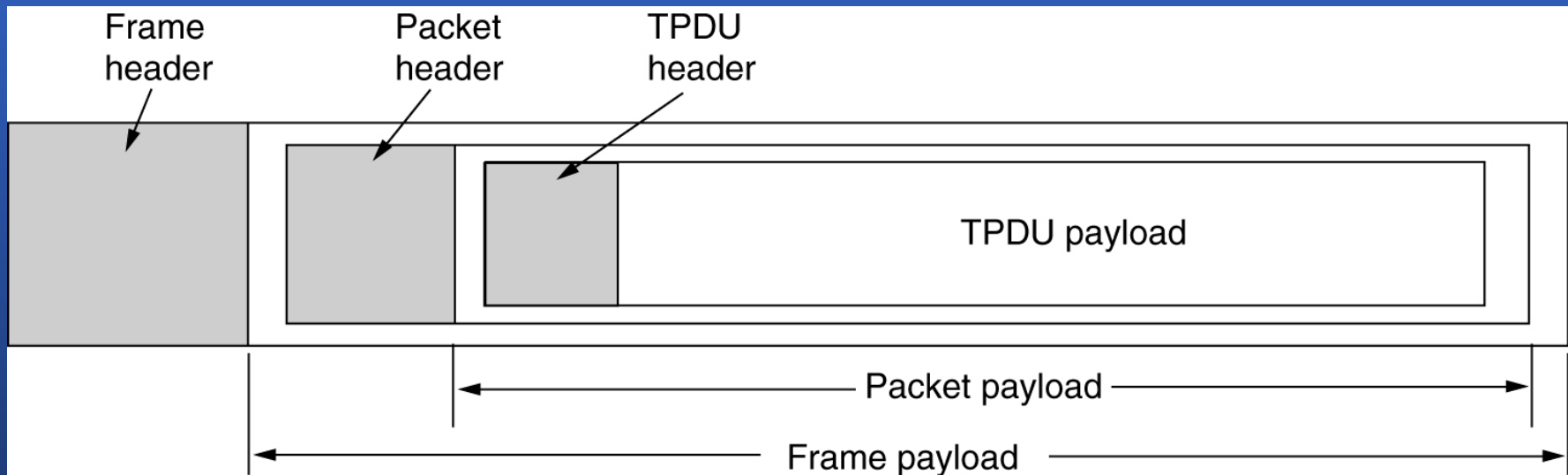
Primitive	Packet sent	Meaning
LISTEN	(none)	Block until some process tries to connect
CONNECT	CONNECTION REQ.	Actively attempt to establish a connection
SEND	DATA	Send information
RECEIVE	(none)	Block until a DATA packet arrives
DISCONNECT	DISCONNECTION REQ.	This side wants to release the connection

The primitives for a simple transport service.



Transport Service Primitives (2)

- 传输协议数据TPDU
- TPDU, 分组和帧的嵌套:

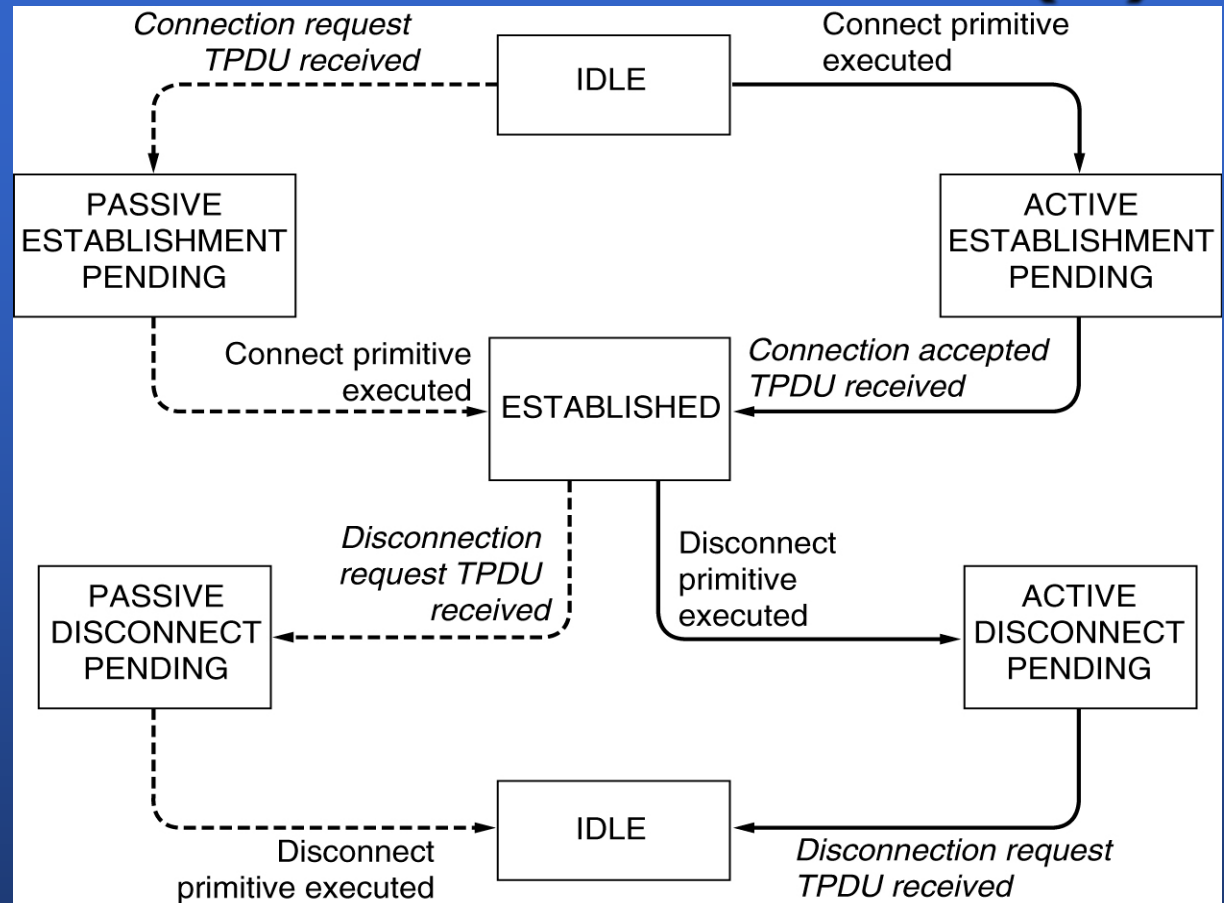


The nesting of TPDUs, packets, and frames.

Transport Service Primitives (3)

- Disconnection

- Asymmetric
- Symmetric



A state diagram for a simple connection management scheme. Transitions labeled in italics are caused by packet arrivals. The solid lines show the client's state sequence. The dashed lines show the server's state sequence.

6.1.3 Berkeley Sockets

Primitive	Meaning
SOCKET	Create a new communication end point
BIND	Attach a local address to a socket
LISTEN	Announce willingness to accept connections; give queue size
ACCEPT	Block the caller until a connection attempt arrives
CONNECT	Actively attempt to establish a connection
SEND	Send some data over the connection
RECEIVE	Receive some data from the connection
CLOSE	Release the connection

The socket primitives for TCP.



6.1.4 Socket Programming

Example: Internet File Server

Client code using sockets.

```
/* This page contains a client program that can request a file from the server program
* on the next page. The server responds by sending the whole file.
*/

#include <sys/types.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <netdb.h>

#define SERVER_PORT 12345          /* arbitrary, but client & server must agree */
#define BUF_SIZE 4096             /* block transfer size */

int main(int argc, char **argv)
{
    int c, s, bytes;
    char buf[BUF_SIZE];             /* buffer for incoming file */
    struct hostent *h;              /* info about server */
    struct sockaddr_in channel;     /* holds IP address */

    if (argc != 3) fatal("Usage: client server-name file-name");
    h = gethostbyname(argv[1]);     /* look up host's IP address */
    if (!h) fatal("gethostbyname failed");

    s = socket(PF_INET, SOCK_STREAM, IPPROTO_TCP);
    if (s < 0) fatal("socket");
    memset(&channel, 0, sizeof(channel));
    channel.sin_family = AF_INET;
    memcpy(&channel.sin_addr.s_addr, h->h_addr, h->h_length);
    channel.sin_port = htons(SERVER_PORT);

    c = connect(s, (struct sockaddr *) &channel, sizeof(channel));
    if (c < 0) fatal("connect failed");

    /* Connection is now established. Send file name including 0 byte at end. */
    write(s, argv[2], strlen(argv[2])+1);

    /* Go get the file and write it to standard output. */
    while (1) {
        bytes = read(s, buf, BUF_SIZE);    /* read from socket */
        if (bytes <= 0) exit(0);           /* check for end of file */
        write(1, buf, bytes);              /* write to standard output */
    }
}

fatal(char *string)
{
    printf("%s\n", string);
    exit(1);
}
```



Socket Programming Example: Internet File Server (2)

Server code using sockets.

```
#include <sys/types.h>
#include <sys/fcntl.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <netdb.h>

#define SERVER_PORT 12345
#define BUF_SIZE 4096
#define QUEUE_SIZE 10

int main(int argc, char *argv[])
{
    int s, b, l, fd, sa, bytes, on = 1;
    char buf[BUF_SIZE];
    struct sockaddr_in channel;

    /* Build address structure to bind to socket. */
    memset(&channel, 0, sizeof(channel)); /* zero channel */
    channel.sin_family = AF_INET;
    channel.sin_addr.s_addr = htonl(INADDR_ANY);
    channel.sin_port = htons(SERVER_PORT);

    /* Passive open. Wait for connection. */
    s = socket(AF_INET, SOCK_STREAM, IPPROTO_TCP); /* create socket */
    if (s < 0) fatal("socket failed");
    setsockopt(s, SOL_SOCKET, SO_REUSEADDR, (char *) &on, sizeof(on));

    b = bind(s, (struct sockaddr *) &channel, sizeof(channel));
    if (b < 0) fatal("bind failed");

    l = listen(s, QUEUE_SIZE); /* specify queue size */
    if (l < 0) fatal("listen failed");

    /* Socket is now set up and bound. Wait for connection and process it. */
    while (1) {
        sa = accept(s, 0, 0); /* block for connection request */
        if (sa < 0) fatal("accept failed");

        read(sa, buf, BUF_SIZE); /* read file name from socket */

        /* Get and return the file. */
        fd = open(buf, O_RDONLY); /* open the file to be sent back */
        if (fd < 0) fatal("open failed");

        while (1) {
            bytes = read(fd, buf, BUF_SIZE); /* read from file */
            if (bytes <= 0) break; /* check for end of file */
            write(sa, buf, bytes); /* write bytes to socket */
        }

        close(fd); /* close file */
        close(sa); /* close connection */
    }
}
```

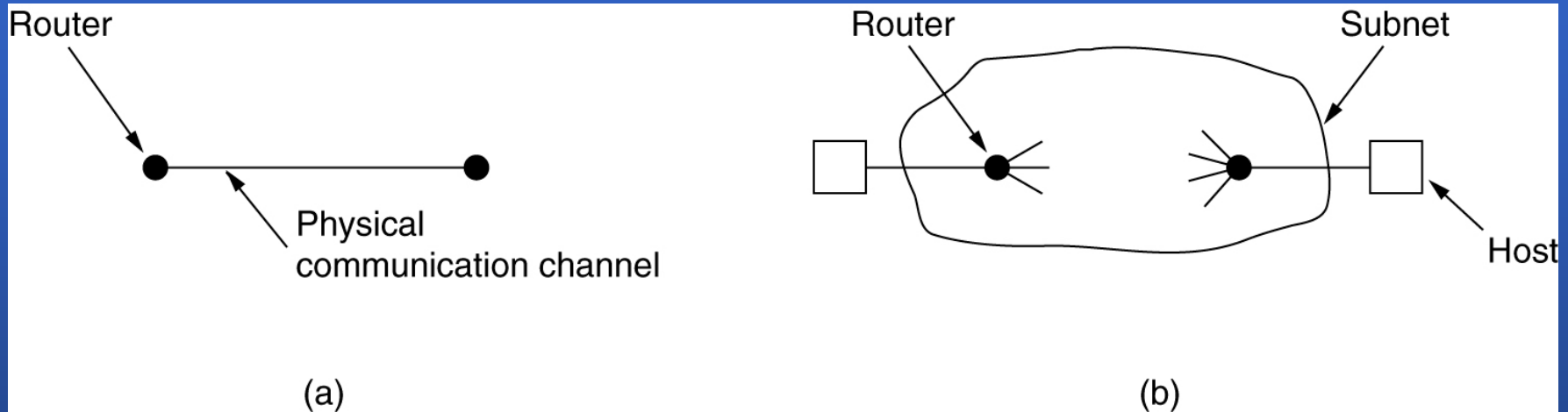


6.2 Elements of Transport Protocols

- Addressing
- Connection Establishment
- Connection Release
- Flow Control and Buffering
- Multiplexing
- Crash Recovery



Transport Protocol



(a) Environment of the data link layer.

(b) Environment of the transport layer.

Transport Protocol

- 与数据链路层的差异

- 两者都要解决：差错控制、分组顺序、流量控制等问题
- 路由、目的地址
- 连接建立过程
- 子网的存储能力
- （数据缓冲、流量控制）数量上的差别

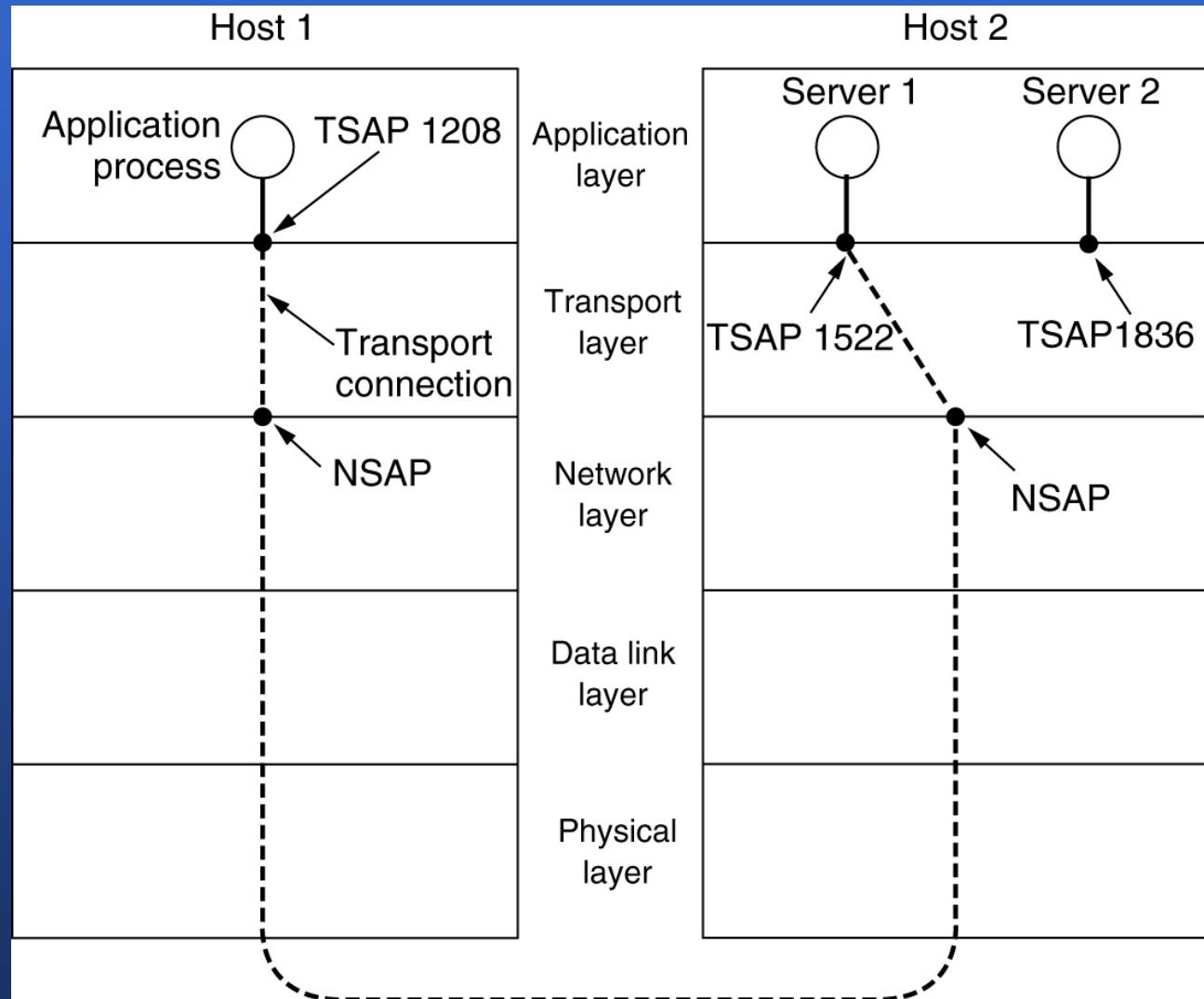


6.2.1 Addressing

Ports: end points

TSAP: Transport Service Access Point

NSAP:



TSAPs, NSAPs and transport connections.

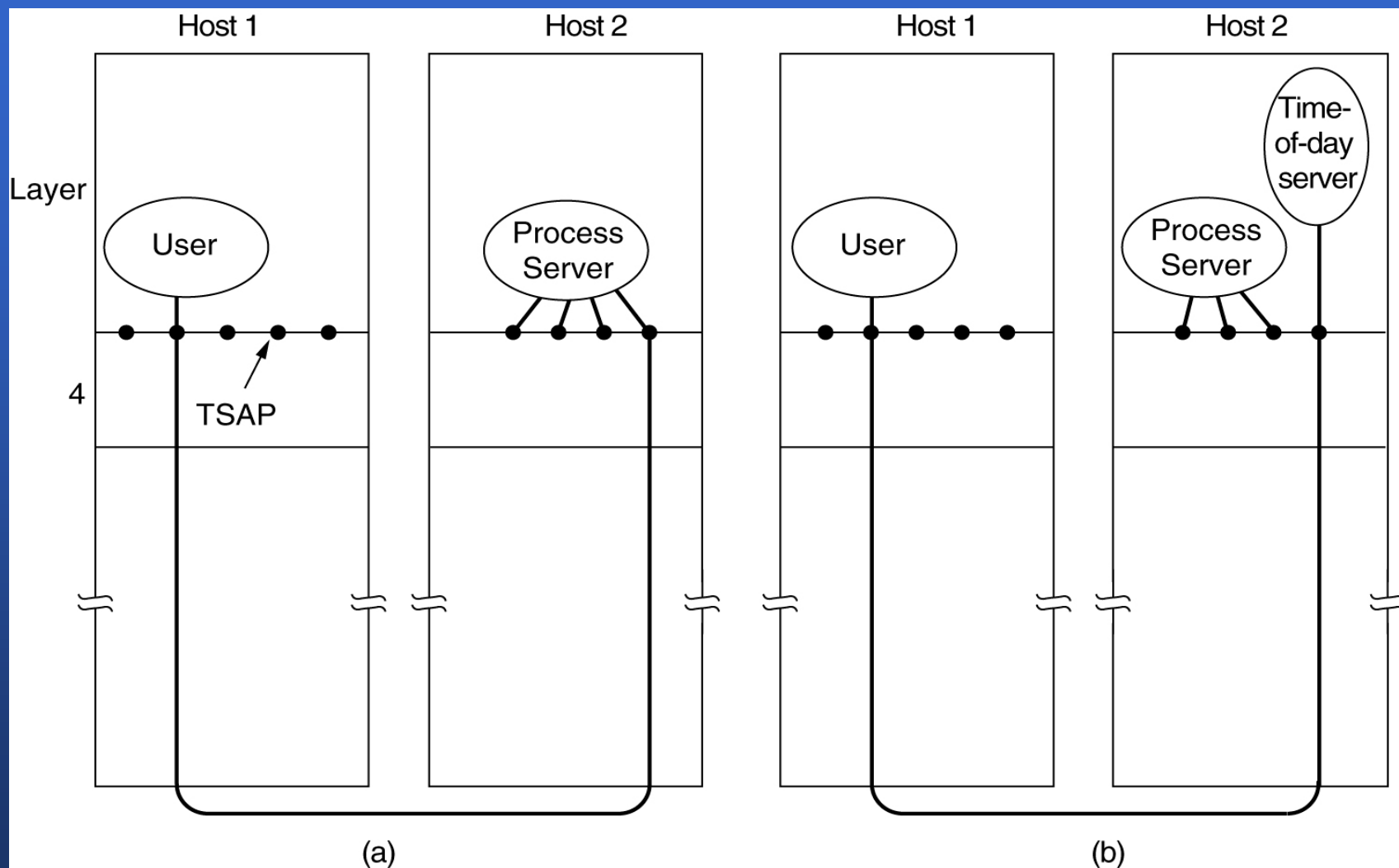


Addressing(2)

- Name Server (Directory Server) :用于查找TSAP地址
- 地址空间
 - 层次型地址
 - 地址=<星系><恒星><行星><国家><网络><主机><端口>
 - 如: +86, 571, 7952114
 - 平面型地址
 - 通过二级映射 (如 DNS服务器)来定位机器



6.2.2 Connection Establishment



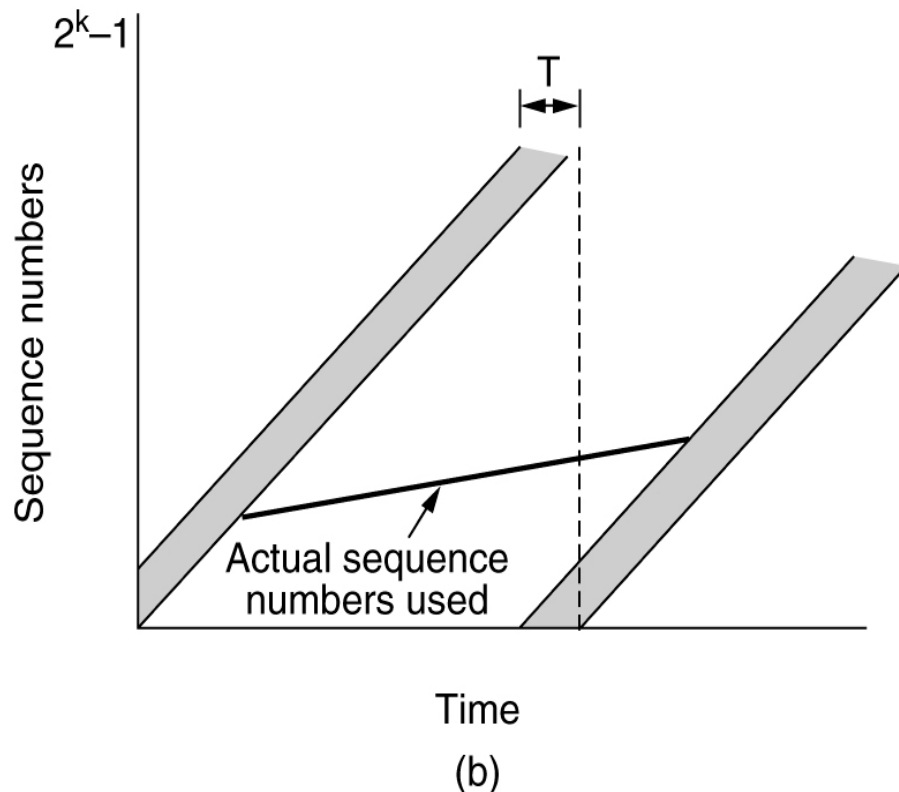
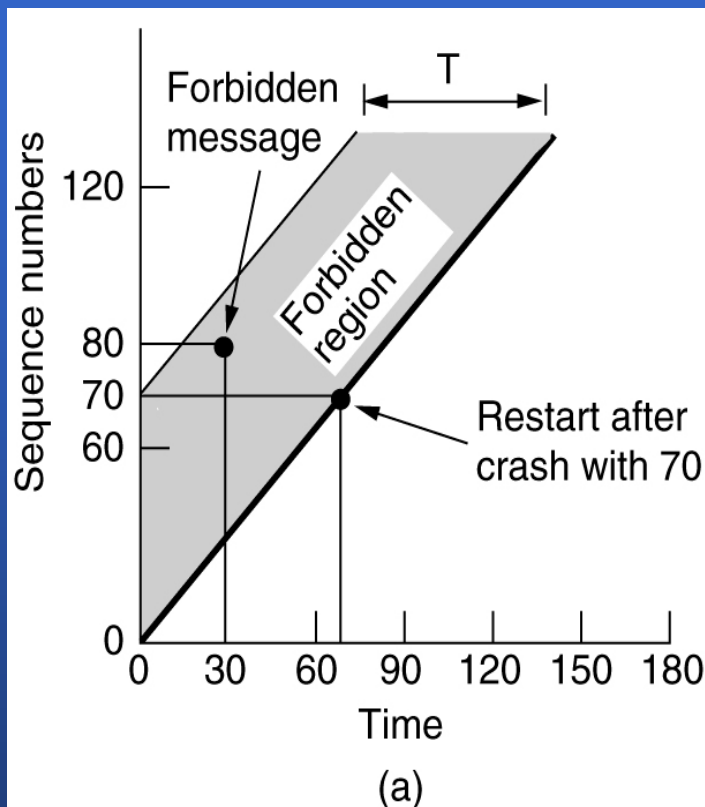
How a user process in host 1 establishes a connection with a time-of-day server in host 2.

Connection Establishment (2)

- 一个复杂的过程：延迟的重复分组的干扰
- 解决方法：
 - 废弃使用过的传输地址
 - 给每个连接分配一个连接标识符
- 缺陷：每个传输实体长时间保存历史信息。如果系统崩溃，将造成信息丢失
- 改进技术，删除在子网中“徘徊”的过时分组：
 - 将分组的生命周期限制在一个已知的最大值内：
 - 受限制的子网设计
 - 在每个分组内设计一个站段计数器
 - 为每个分组加上时间标记



Connection Establishment (3)

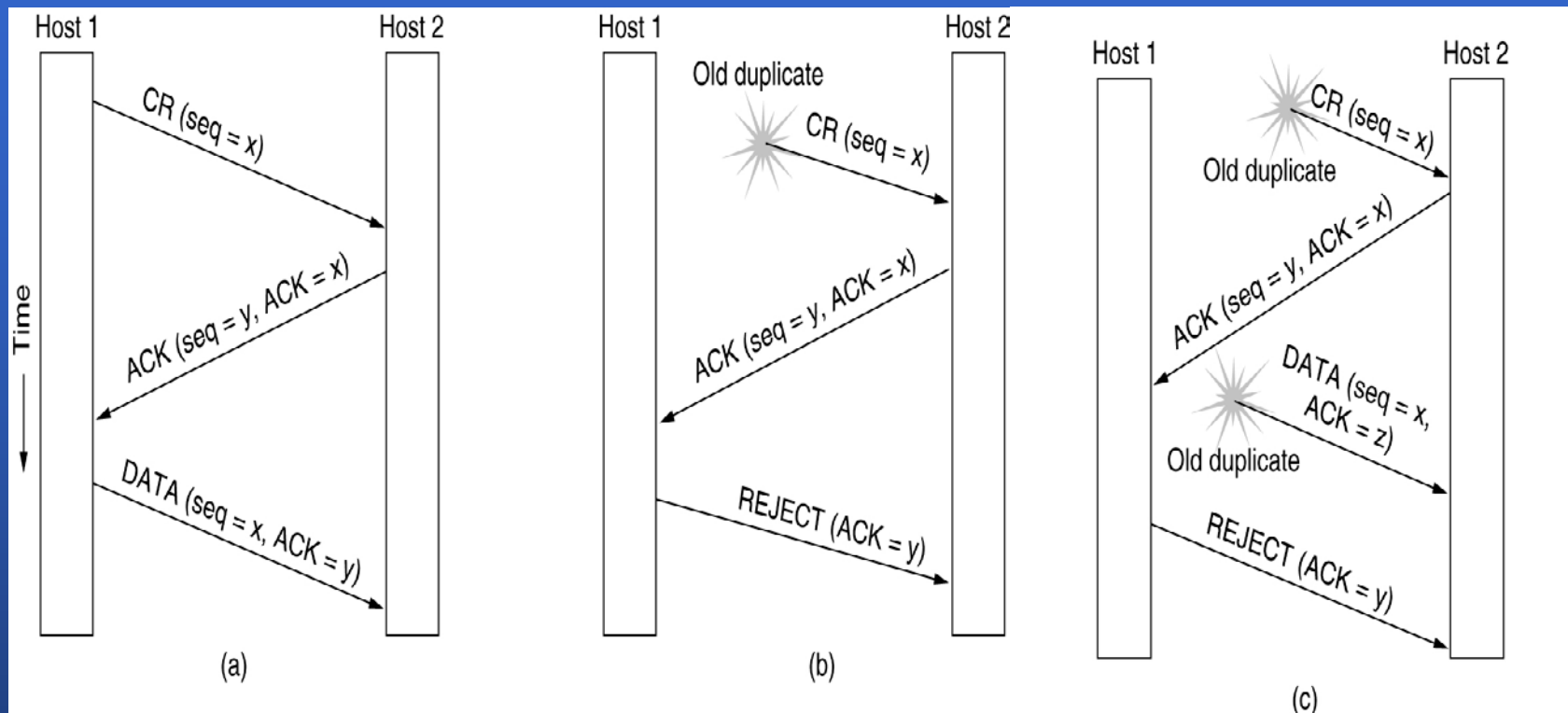


(a) TPDUs may not enter the forbidden region.

(b) The resynchronization problem.

基本思想：确保在同一时刻永远不会出现两个相同编号的TPDU

Connection Establishment (4)

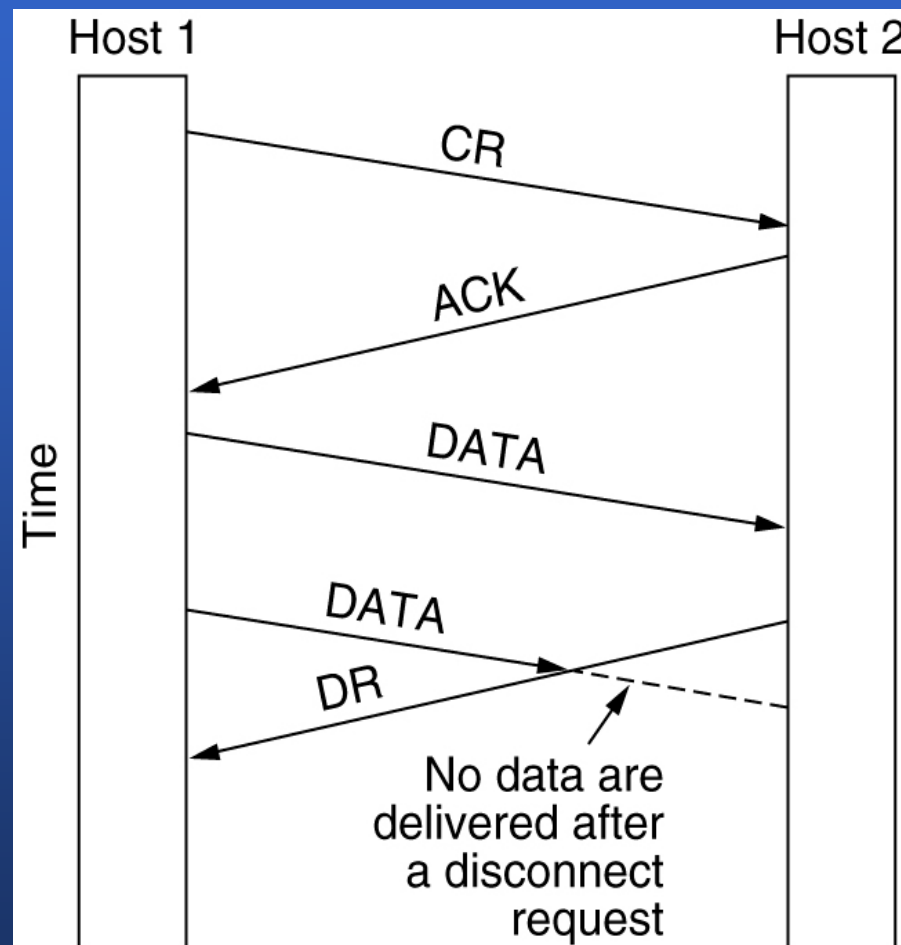


Three protocol scenarios for establishing a connection using a **three-way handshake**. CR denotes CONNECTION REQUEST.

- (a) Normal operation,
- (b) Old CONNECTION REQUEST appearing out of nowhere.
- (c) Duplicate CONNECTION REQUEST and duplicate ACK.

6.2.3 Connection Release

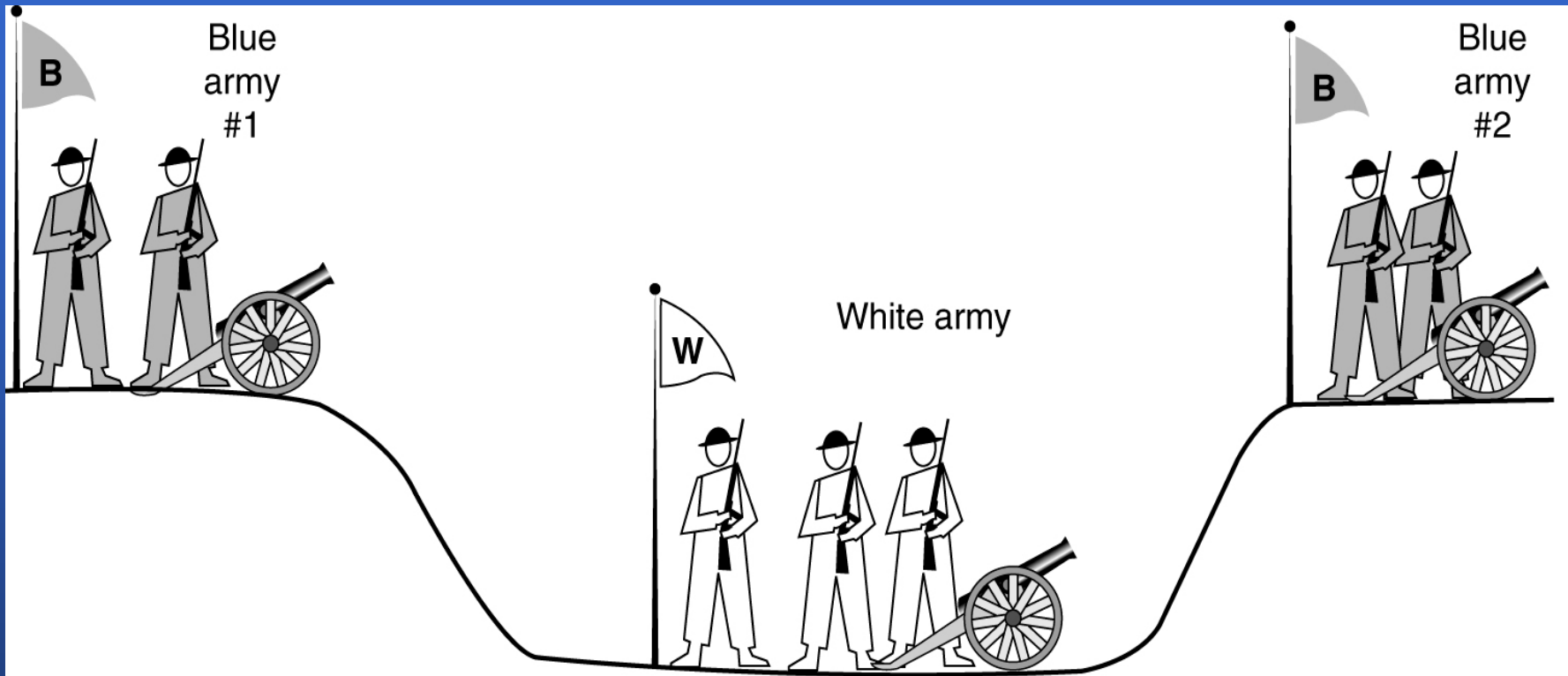
- 终止连接的两种方法:
 - 非对称释放——可能导致丢失数据—[see fig 6-12]
 - 对称释放——适用于固定数量的数据发送，且知道何时完毕



Abrupt disconnection with loss of data.



Connection Release (2)



The two-army problem:

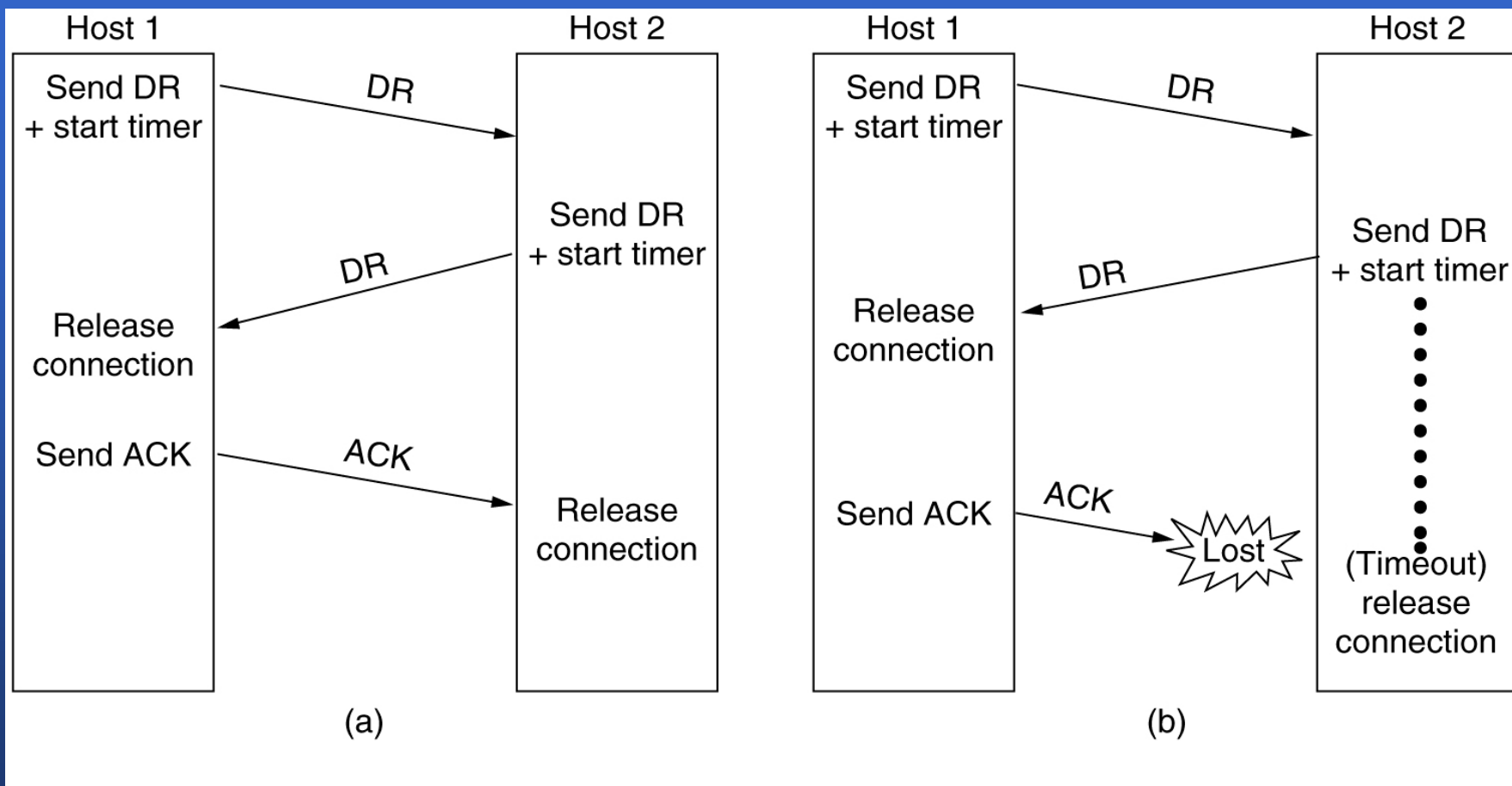
无论三次、四次…的握手都不能圆满地解决问题

Connection Release (3)

- 实际应用，三次握手-[see fig 6-14]
 - 并非绝对无误，但却令人满意
 - 三次握手的不足：第一个DR和所有N次的重发，最终造成**半连通**的连接 → 协议失败
 - **消除半连通的办法**：在一段时间内没有受到任何TPDU，便自动释放连接

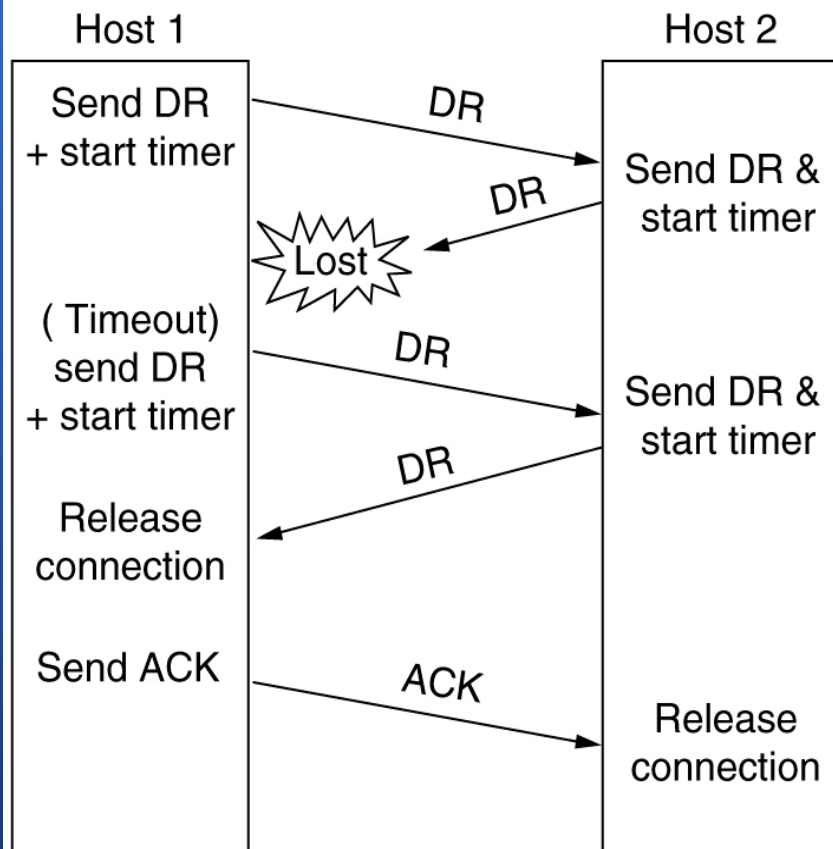


Connection Release (4)

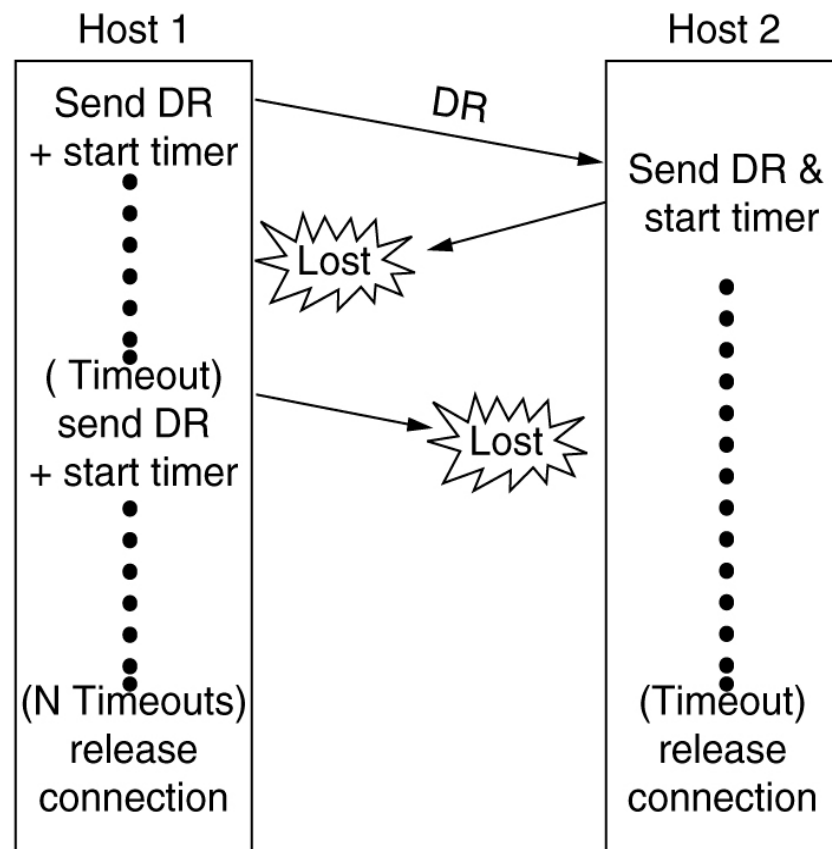


Four protocol scenarios for releasing a connection. (a) Normal case of a three-way handshake. (b) final ACK lost.

Connection Release (5)



(c)



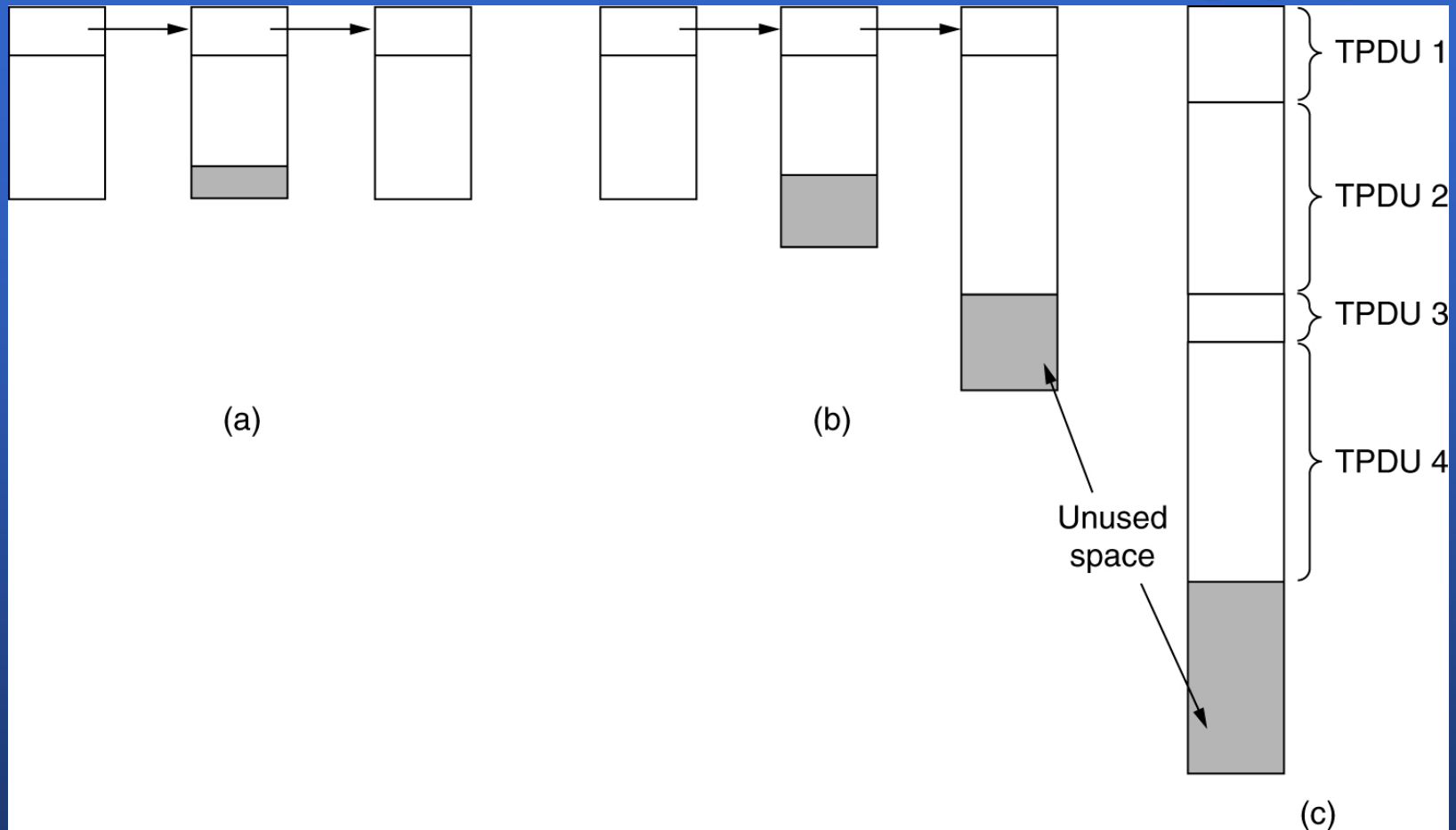
(d)

(c) Response lost. (d) Response lost and subsequent DRs lost.

6.2.4 Flow Control and Buffering[*]

- 考虑连接在使用过程中的管理问题
- 通过滑动窗口和其它机制
- 数据缓冲的方法： - [see fig 6-15]
 - 固定大小
 - 可变大小
 - 循环缓冲区
- 另一个可能的瓶颈：子网的运载容量

Flow Control and Buffering(2)



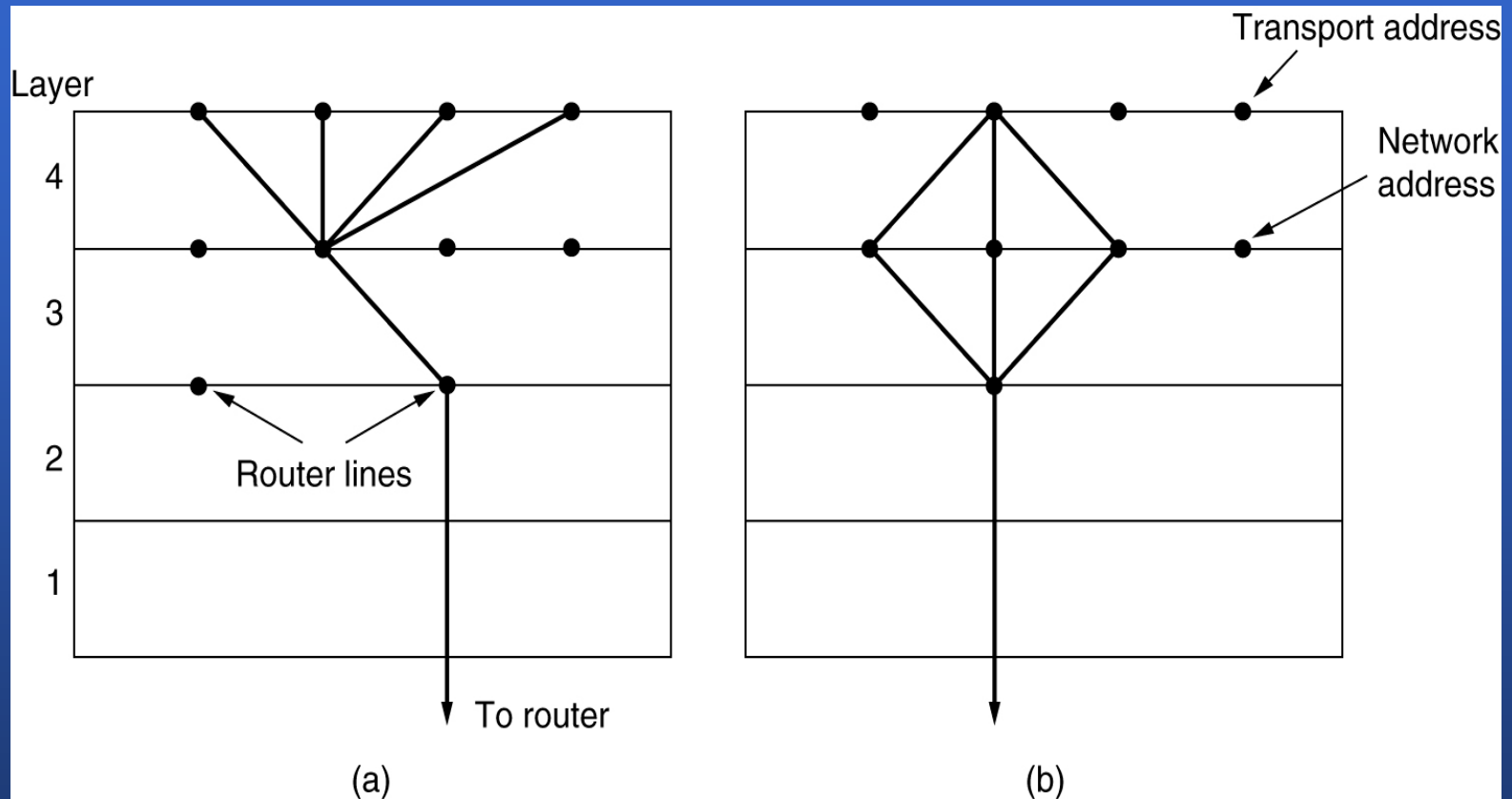
- (a) Chained fixed-size buffers. (b) Chained variable-sized buffers.
(c) One large circular buffer per connection.

Flow Control and Buffering (3)

	<u>A</u>	<u>Message</u>	<u>B</u>	<u>Comments</u>
1	→	< request 8 buffers>	→	A wants 8 buffers
2	←	<ack = 15, buf = 4>	←	B grants messages 0-3 only
3	→	<seq = 0, data = m0>	→	A has 3 buffers left now
4	→	<seq = 1, data = m1>	→	A has 2 buffers left now
5	→	<seq = 2, data = m2>	...	Message lost but A thinks it has 1 left
6	←	<ack = 1, buf = 3>	←	B acknowledges 0 and 1, permits 2-4
7	→	<seq = 3, data = m3>	→	A has 1 buffer left
8	→	<seq = 4, data = m4>	→	A has 0 buffers left, and must stop
9	→	<seq = 2, data = m2>	→	A times out and retransmits
10	←	<ack = 4, buf = 0>	←	Everything acknowledged, but A still blocked
11	←	<ack = 4, buf = 1>	←	A may now send 5
12	←	<ack = 4, buf = 2>	←	B found a new buffer somewhere
13	→	<seq = 5, data = m5>	→	A has 1 buffer left
14	→	<seq = 6, data = m6>	→	A is now blocked again
15	←	<ack = 6, buf = 0>	←	A is still blocked
16	...	<ack = 6, buf = 4>	←	Potential deadlock

Dynamic buffer allocation. The arrows show the direction of transmission. An ellipsis (...) indicates a lost TPDU.

6.2.5 Multiplexing [*]



(a) Upward multiplexing: 考虑连接的费用问题

(b) Downward multiplexing: 考虑连接的技术（带宽、可靠性）问题, 以提高连接的带宽

6.2.6 Crash Recovery [*]

- 从网络和路由器中恢复
 - 如果网络层提供数据报服务，传输实体留有丢失的TPDU的副本
 - 如果网络层提供面向连接的服务，重建新的虚电路，重发丢失的TPDU
- 从主机或服务器崩溃中恢复，存在着复杂的问题-
[see fig 6-18]
- 结论：从第N层崩溃中恢复只能由第N+1层来完成，并且只能由第N+1层留有足够状态信息的情况下才能完成



Crash Recovery(2)

		Strategy used by receiving host					
		First ACK, then write			First write, then ACK		
Strategy used by sending host		AC(W)	AWC	C(AW)	C(WA)	W AC	WC(A)
Always retransmit		OK	DUP	OK	OK	DUP	DUP
Never retransmit		LOST	OK	LOST	LOST	OK	OK
Retransmit in S0		OK	DUP	LOST	LOST	DUP	OK
Retransmit in S1		LOST	OK	OK	OK	OK	DUP

OK = Protocol functions correctly
 DUP = Protocol generates a duplicate message
 LOST = Protocol loses a message

Different combinations of client and server strategy.

A:Acknowledgement, **W**: writing to the output process, **C**: crashing

6.3 A Simple Transport Protocol [*]

- The Example Service Primitives
- The Example Transport Entity
- The Example as a Finite State Machine



6.3.1 The Example Service Primitives

- 5条原语
 - Connum = LISTEN(local)
 - Connum = CONNECT(local,remote)
 - Status = SEND(connum,buffer,bytes)
 - Status = RECEIVE(connum,buffer,bytes)
 - Status = DISCONNECT(connum)



6.3.2 The Example Transport Entity

- 选用面向连接的网络服务
- 与网络层的接口: to_net 和 from_net
 - 网络层分组类型-[see fig 6-19]
- 网络层通过 enable_transport_layer 和 disable_transport_layer 控制传输层对其调用

Network packet	Meaning
CALL REQUEST	Sent to establish a connection
CALL ACCEPTED	Response to CALL REQUEST
CLEAR REQUEST	Sent to release a connection
CLEAR CONFIRMATION	Response to CLEAR REQUEST
DATA	Used to transport data
CREDIT	Control packet for managing the window

The network layer packets used in our example.



The Example Transport Entity (2)

Each connection is in one of seven states:

- IDLE – Connection not established yet.
- WAITING – CONNECT has been executed, CALL REQUEST sent.
- QUEUED – A CALL REQUEST has arrived; no LISTEN yet.
- ESTABLISHED – The connection has been established.
- SENDING – The user is waiting for permission to send a packet.
- RECEIVING – A RECEIVE has been done.
- DISCONNECTING – a DISCONNECT has been done locally.

Main data structure: conn



The Example Transport Entity (3)

```
#define MAX_CONN 32                /* max number of simultaneous connections */
#define MAX_MSG_SIZE 8192          /* largest message in bytes */
#define MAX_PKT_SIZE 512          /* largest packet in bytes */
#define TIMEOUT 20
#define CRED 1
#define OK 0

#define ERR_FULL -1
#define ERR_REJECT -2
#define ERR_CLOSED -3
#define LOW_ERR -3

typedef int transport_address;
typedef enum {CALL_REQ,CALL_ACC,CLEAR_REQ,CLEAR_CONF,DATA_PKT,CREDIT} pkt_type;
typedef enum {IDLE,WAITING,QUEUED,ESTABLISHED,SENDING,RECEIVING,DISCONN} cstate;

/* Global variables. */
transport_address listen_address; /* local address being listened to */
int listen_conn;                 /* connection identifier for listen */
unsigned char data[MAX_PKT_SIZE]; /* scratch area for packet data */

struct conn {
    transport_address local_address, remote_address;
    cstate state; /* state of this connection */
    unsigned char *user_buf_addr; /* pointer to receive buffer */
    int byte_count; /* send/receive count */
    int clr_req_received; /* set when CLEAR_REQ packet received */
    int timer; /* used to time out CALL_REQ packets */
    int credits; /* number of messages that may be sent */
} conn[MAX_CONN + 1]; /* slot 0 is not used */
```

The Example Transport Entity (4)

```
void sleep(void);                /* prototypes */
void wakeup(void);
void to_net(int cid, int q, int m, pkt_type pt, unsigned char *p, int bytes);
void from_net(int *cid, int *q, int *m, pkt_type *pt, unsigned char *p, int *bytes);

int listen(transport_address t)
{ /* User wants to listen for a connection. See if CALL_REQ has already arrived. */
  int i, found = 0;

  for (i = 1; i <= MAX_CONN; i++)          /* search the table for CALL_REQ */
    if (conn[i].state == QUEUED && conn[i].local_address == t) {
      found = i;
      break;
    }

  if (found == 0) {
    /* No CALL_REQ is waiting. Go to sleep until arrival or timeout. */
    listen_address = t; sleep(); i = listen_conn ;
  }

  conn[i].state = ESTABLISHED;              /* connection is ESTABLISHED */
  conn[i].timer = 0;                        /* timer is not used */
}
```

The Example Transport Entity (5)

```
listen_conn = 0; /* 0 is assumed to be an invalid address */
to_net(i, 0, 0, CALL_ACC, data, 0); /* tell net to accept connection */
return(i); /* return connection identifier */
}

int connect(transport_address l, transport_address r)
{ /* User wants to connect to a remote process; send CALL_REQ packet. */
  int i;
  struct conn *cptr;

  data[0] = r; data[1] = l; /* CALL_REQ packet needs these */
  i = MAX_CONN; /* search table backward */
  while (conn[i].state != IDLE && i > 1) i = i - 1;
  if (conn[i].state == IDLE) {
    /* Make a table entry that CALL_REQ has been sent. */
    cptr = &conn[i];
    cptr->local_address = l; cptr->remote_address = r;
    cptr->state = WAITING; cptr->clr_req_received = 0;
    cptr->credits = 0; cptr->timer = 0;
    to_net(i, 0, 0, CALL_REQ, data, 2);
    sleep(); /* wait for CALL_ACC or CLEAR_REQ */
    if (cptr->state == ESTABLISHED) return(i);
    if (cptr->clr_req_received) {
      /* Other side refused call. */
      cptr->state = IDLE; /* back to IDLE state */
      to_net(i, 0, 0, CLEAR_CONF, data, 0);
      return(ERR_REJECT);
    }
  }
  else return(ERR_FULL); /* reject CONNECT: no table space */
}
```

The Example Transport Entity (6)

```
int send(int cid, unsigned char bufptr[], int bytes)
{ /* User wants to send a message. */
    int i, count, m;
    struct conn *cptr = &conn[cid];

    /* Enter SENDING state. */
    cptr->state = SENDING;
    cptr->byte_count = 0; /* # bytes sent so far this message */
    if (cptr->clr_req_received == 0 && cptr->credits == 0) sleep();
    if (cptr->clr_req_received == 0) {
        /* Credit available; split message into packets if need be. */
        do {
            if (bytes - cptr->byte_count > MAX_PKT_SIZE) { /* multipacket message */
                count = MAX_PKT_SIZE; m = 1; /* more packets later */
            } else { /* single packet message */
                count = bytes - cptr->byte_count; m = 0; /* last pkt of this message */
            }
            for (i = 0; i < count; i++) data[i] = bufptr[cptr->byte_count + i];
            to_net(cid, 0, m, DATA_PKT, data, count); /* send 1 packet */
            cptr->byte_count = cptr->byte_count + count; /* increment bytes sent so far */
        } while (cptr->byte_count < bytes); /* loop until whole message sent */
    }
```

The Example Transport Entity (7)

```
    cptr->credits --;                                /* * each message uses up one credit */
    cptr->state = ESTABLISHED;
    return(OK);
} else {
    cptr->state = ESTABLISHED;
    return(ERR_CLOSED);                               /* send failed: peer wants to disconnect */
}
}
```

```
int receive(int cid, unsigned char bufptr[], int *bytes)
```

```
{ /* User is prepared to receive a message. */
```

```
    struct conn *cptr = &conn[cid];
```

```
    if (cptr->clr_req_received == 0) {
```

```
        /* Connection still established; try to receive. */
```

```
        cptr->state = RECEIVING;
```

```
        cptr->user_buf_addr = bufptr;
```

```
        cptr->byte_count = 0;
```

```
        data[0] = CRED;
```

```
        data[1] = 1;
```

```
        to_net(cid, 1, 0, CREDIT, data, 2);
```

```
        /* send credit */
```

```
        sleep();
```

```
        /* block awaiting data */
```

```
        *bytes = cptr->byte_count;
```

```
    }
```

```
    cptr->state = ESTABLISHED;
```

```
    return(cptr->clr_req_received ? ERR_CLOSED : OK);
```

信用报文 (Credit Message)

The Example Transport Entity (8)

```
int disconnect(int cid)
{ /* User wants to release a connection. */
  struct conn *cptr = &conn[cid];

  if (cptr->clr_req_received) { /* other side initiated termination */
    cptr->state = IDLE; /* connection is now released */
    to_net(cid, 0, 0, CLEAR_CONF, data, 0);
  } else { /* we initiated termination */
    cptr->state = DISCONN; /* not released until other side agrees */
    to_net(cid, 0, 0, CLEAR_REQ, data, 0);
  }
  return(OK);
}

void packet_arrival(void)
{ /* A packet has arrived, get and process it. */
  int cid; /* connection on which packet arrived */
  int count, i, q, m;
  pkt_type ptype; /* CALL_REQ, CALL_ACC, CLEAR_REQ, CLEAR_CONF, DATA_PKT, CREDIT */
  unsigned char data[MAX_PKT_SIZE]; /* data portion of the incoming packet */
  struct conn *cptr;

  from_net(&cid, &q, &m, &ptype, data, &count); /* go get it */
  cptr = &conn[cid];
```


The Example Transport Entity (9)

```
switch (ptype) {
case CALL_REQ:                                /* remote user wants to establish connection */
    cptr->local_address = data[0]; cptr->remote_address = data[1];
    if (cptr->local_address == listen_address) {
        listen_conn = cid; cptr->state = ESTABLISHED; wakeup();
    } else {
        cptr->state = QUEUED; cptr->timer = TIMEOUT;
    }
    cptr->clr_req_received = 0; cptr->credits = 0;
    break;
case CALL_ACC:                                /* remote user has accepted our CALL_REQ */
    cptr->state = ESTABLISHED;
    wakeup();
    break;
case CLEAR_REQ:                                /* remote user wants to disconnect or reject call */
    cptr->clr_req_received = 1;
    if (cptr->state == DISCONN) cptr->state = IDLE; /* clear collision */
    if (cptr->state == WAITING || cptr->state == RECEIVING || cptr->state == SENDING) wakeup();
    break;
case CLEAR_CONF:                                /* remote user agrees to disconnect */
    cptr->state = IDLE;
    break;
case CREDIT:                                    /* remote user is waiting for data */
    cptr->credits += data[1];
    if (cptr->state == SENDING) wakeup();
    break;
case DATA_PKT:                                /* remote user has sent data */
    for (i = 0; i < count; i++) cptr->user_buf_addr[cptr->byte_count + i] = data[i];
    cptr->byte_count += count;
    if (m == 0) wakeup();
}
```

The Example Transport Entity (10)

```
}  
  
void clock(void)  
{ /* The clock has ticked, check for timeouts of queued connect requests. */  
  int i;  
  struct conn *cptr;  
  
  for (i = 1; i <= MAX_CONN; i++) {  
    cptr = &conn[i];  
    if (cptr->timer > 0) { /* timer was running */  
      cptr->timer--;  
      if (cptr->timer == 0) { /* timer has now expired */  
        cptr->state = IDLE;  
        to_net(i, 0, 0, CLEAR_REQ, data, 0);  
      }  
    }  
  }  
}
```



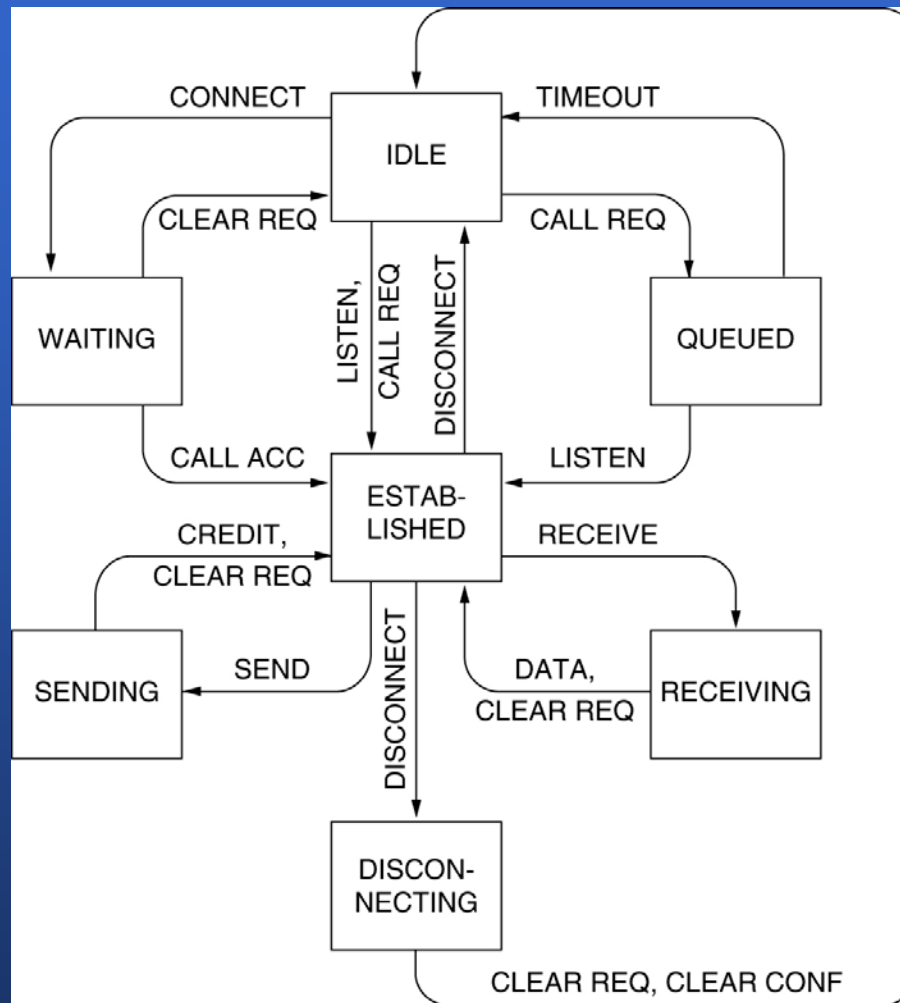
6.3.3 The Example as a Finite State Machine(*)

The example protocol as a finite state machine. Each entry has an optional predicate, an optional action, and the new state. The tilde indicates that no major action is taken. An overbar above a predicate indicate the negation of the predicate. Blank entries correspond to impossible or invalid events.

		State						
		Idle	Waiting	Queued	Established	Sending	Receiving	Dis- connecting
Primitives	LISTEN	P1: ~/Idle P2: A1/Estab P2: A2/Idle		~/Estab				
	CONNECT	P1: ~/Idle P1: A3/Wait						
	DISCONNECT				P4: A5/Idle P4: A6/Disc			
	SEND				P5: A7/Estab P5: A8/Send			
	RECEIVE				A9/Receiving			
Incoming packets	Call_req	P3: A1/Estab P3: A4/Queue'd						
	Call_acc		~/Estab					
	Clear_req		~/Idle		A10/Estab	A10/Estab	A10/Estab	~/Idle
	Clear_conf							~/Idle
	DataPkt						A12/Estab	
Clock	Credit				A11/Estab	A7/Estab		
	Timeout			~/Idle				

Predicates	Actions
P1: Connection table full	A1: Send Call_acc
P2: Call_req pending	A2: Wait for Call_req
P3: LISTEN pending	A3: Send Call_req
P4: Clear_req pending	A4: Start timer
P5: Credit available	A5: Send Clear_conf
	A6: Send Clear_req
	A7: Send message
	A8: Wait for credit
	A9: Send credit
	A10: Set Clr_req_received flag
	A11: Record credit
	A12: Accept message

The Example as a Finite State Machine (2)



The example protocol in graphical form. Transitions that leave the connection state unchanged have been omitted for simplicity.



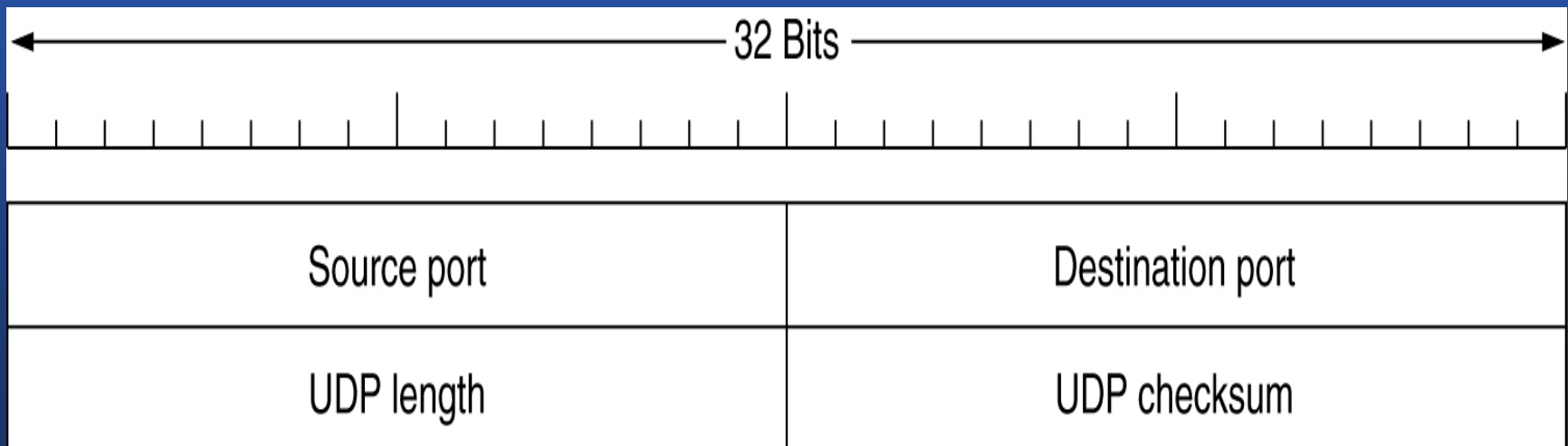
6.4 The Internet Transport Protocols: UDP

- Introduction to UDP
- Remote Procedure Call
- The Real-Time Transport Protocol



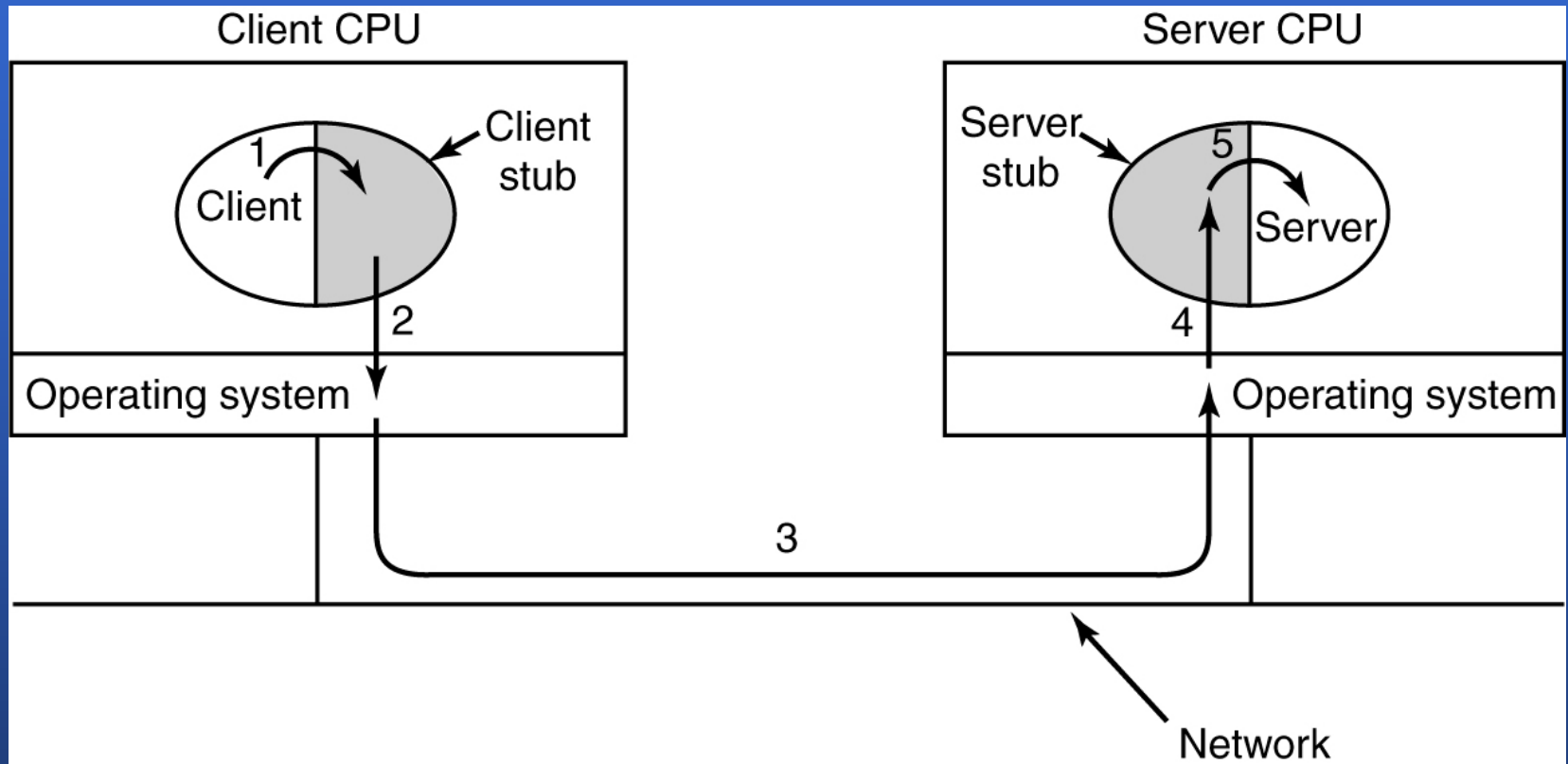
6.4.1 Introduction to UDP

- 无连接的传输协议UDP (User Data Protocol)
 - 发送封装的原始IP数据报
 - 发送时无需建立连接
- UDP数据段包括：8字节头+数据部分
 - 头格式-[see fig 6-23]



The UDP header.

6.4.2 Remote Procedure Call



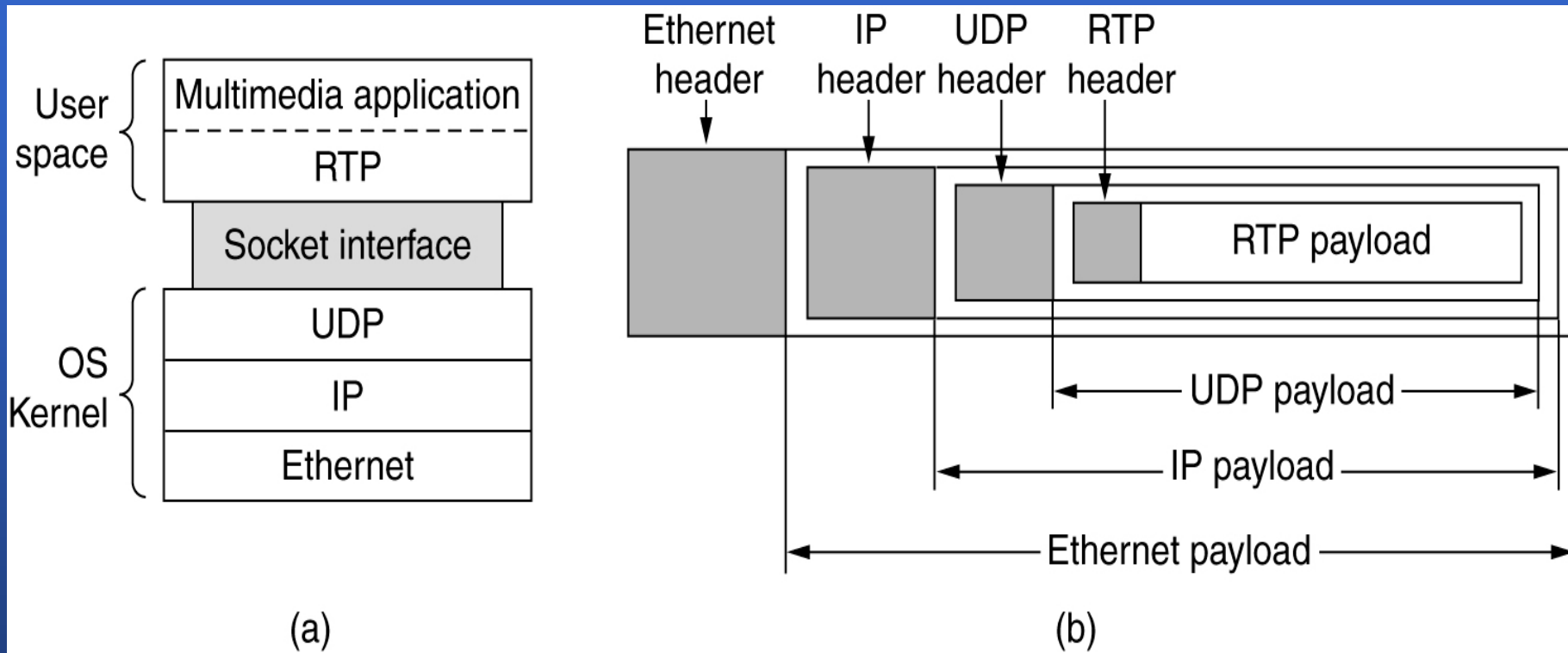
Steps in making a remote procedure call. The stubs are shaded.

6.4.3 The Real-Time Transport Protocol

- **Real-time multimedia applications:**
 - Internet radio
 - Internet telephony
 - Music-on-demand
 - Videoconferencing
 - Video-on-demand
 - ...
- **Basic function of RTP:**
 - Multiplex several real-time data streams onto a single stream of UDP packets
- No flow control, no error control, no acknowledgement and no retransmissions
- **Timestamping:** to make multi-streams play back synchronously

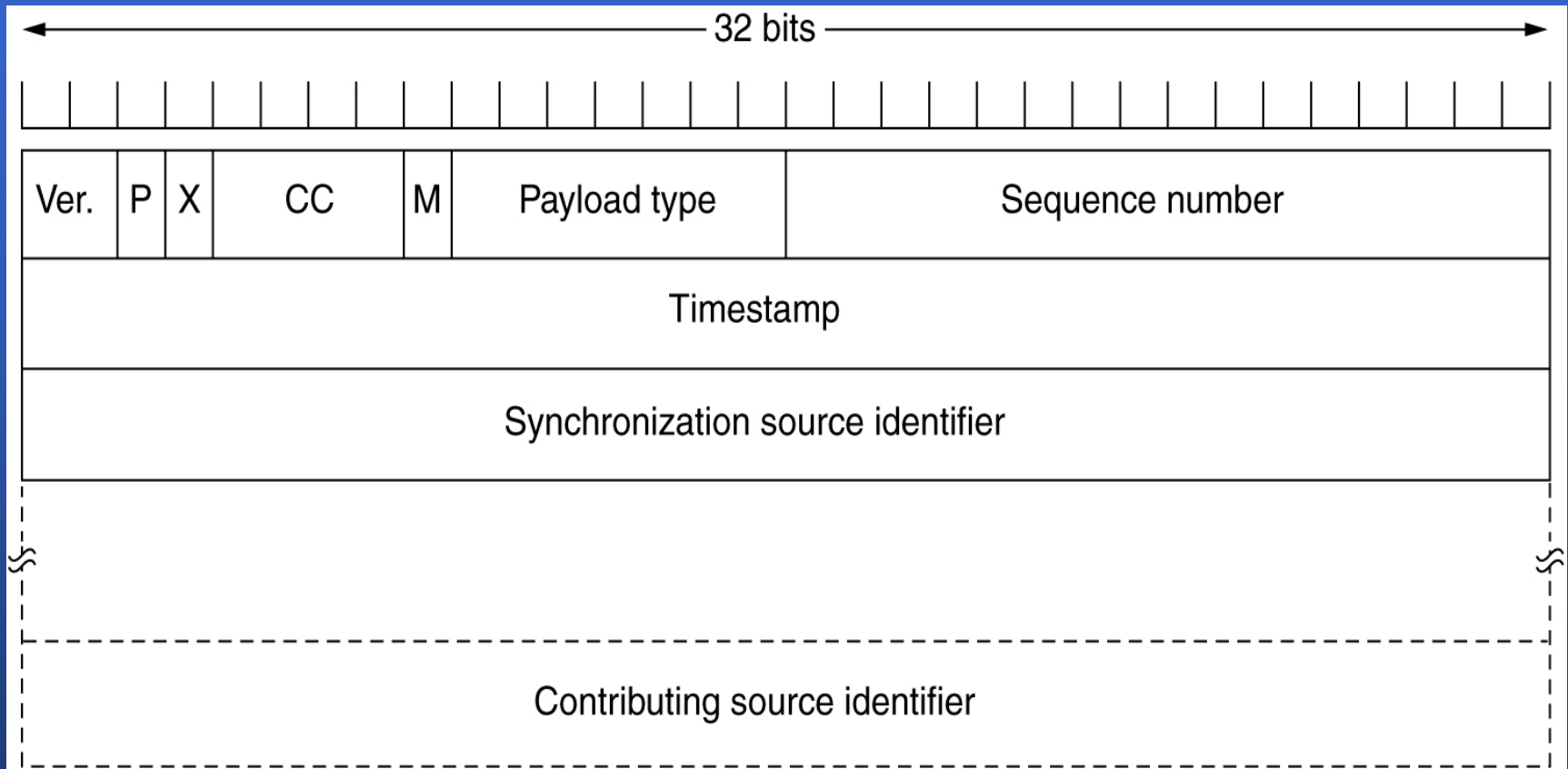


The Real-Time Transport Protocol(2)



- (a) The position of RTP in the protocol stack.
- (b) Packet nesting.

The Real-Time Transport Protocol (3)



The RTP header.

The Real-Time Transport Protocol (4)

- **RTCP** (Realtime Transport Control Protocol)
 - Handle feedback, synchronization



6.5 The Internet Transport Protocols: TCP

- Introduction to TCP
- The TCP Service Model
- The TCP Protocol
- The TCP Segment Header
- TCP Connection Establishment
- TCP Connection Release
- TCP Connection Management Modeling
- TCP Transmission Policy
- TCP Congestion Control
- TCP Timer Management
- Wireless TCP and UDP
- Transactional TCP



6.5.1 Introduction to TCP

- TCP—Transmission Control Protocol
 - Provide a reliable end-to-end byte stream over an unreliable internetwork
 - Connection-oriented
 - Defined in RFC 793, 1122,1323
 - Segment <64KB



6.5.2 The TCP Service Model

- 套接字 (socket)
- 地址: IP地址+端口(port, 16bits)
- Well-known port (通用端口)
 - Port number <1024
 - FTP: 21, Telnet: 23, SMTP: 25 ...
- TCP连接: 全双工、点到点、不支持广播、字节流
- TCP数据的缓存发送和立即发送(带PUSH标志)
- 紧急数据—带URGENT标志



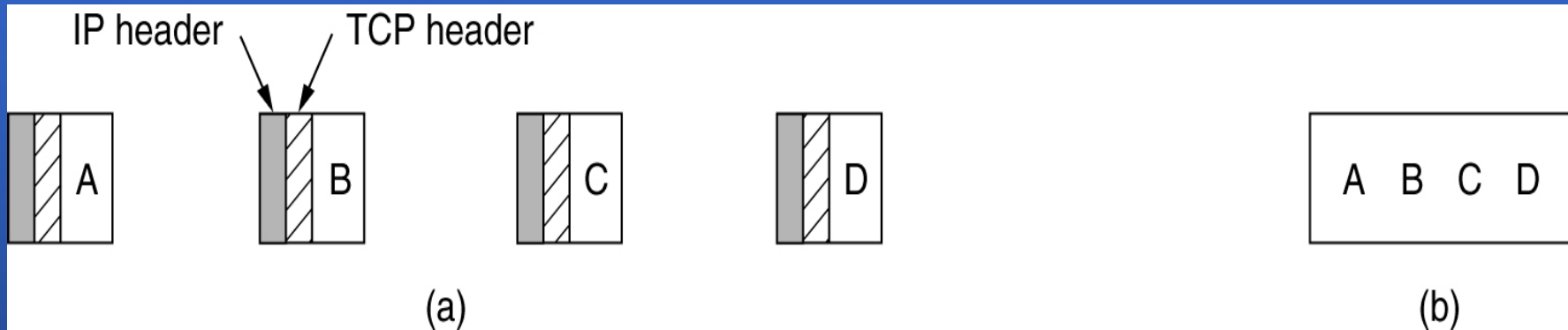
The TCP Service Model(2)

Port	Protocol	Use
21	FTP	File transfer
23	Telnet	Remote login
25	SMTP	E-mail
69	TFTP	Trivial File Transfer Protocol
79	Finger	Lookup info about a user
80	HTTP	World Wide Web
110	POP-3	Remote e-mail access
119	NNTP	USENET news

Some assigned ports.



The TCP Service Model (3)



(a) Four 512-byte segments sent as separate IP datagrams.

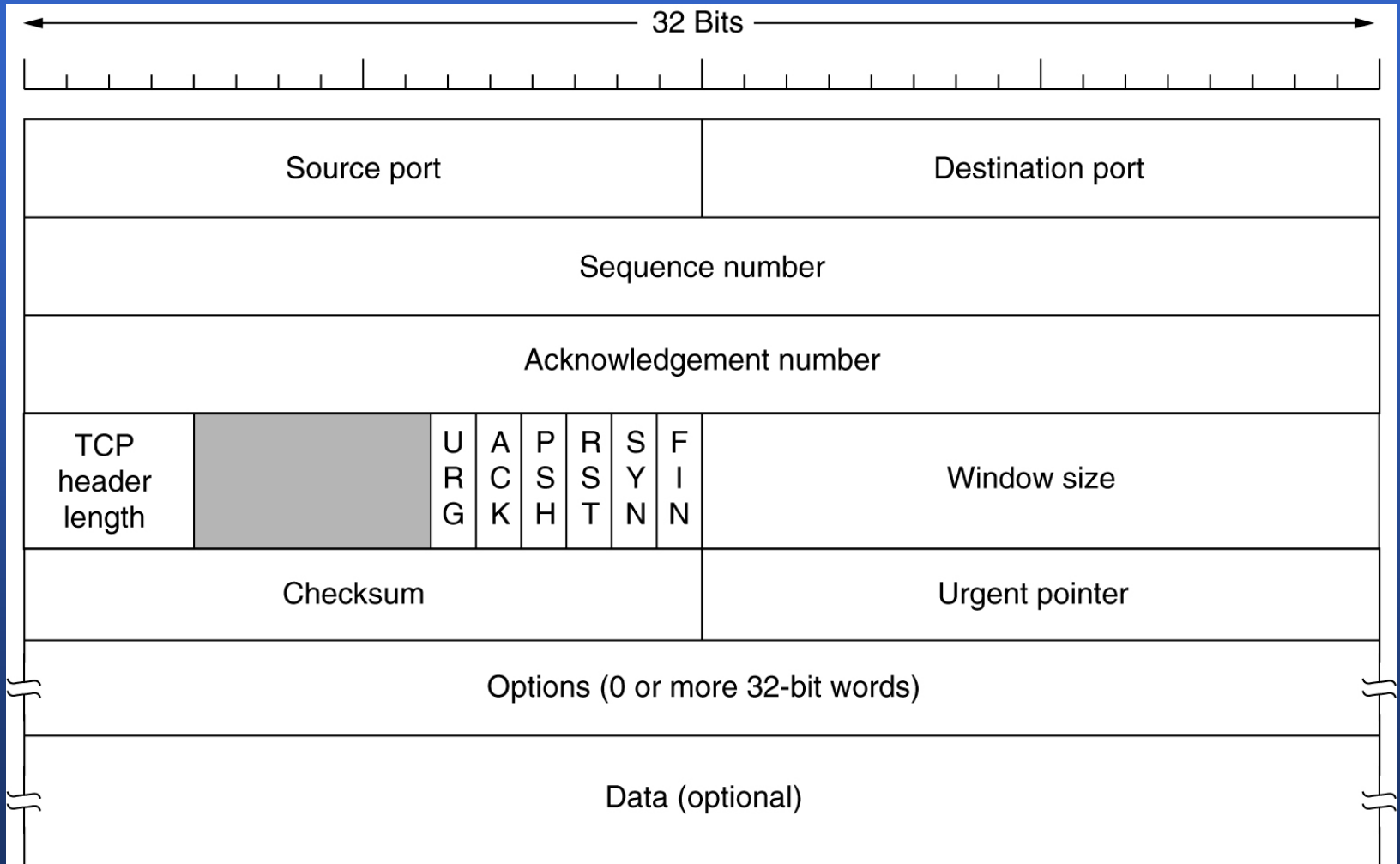
(b) The 2048 bytes of data delivered to the application in a single READ CALL.

6.5.4 The TCP Segment Header

- 数据交换形式：数据段（segment）
- 包含：20字节头+<optional>+<0-n字节数据>
- 大小限制：
 - Segment max size < (65515 字节 and MTU)
 - $65535 - 20 \text{ (IP Header)} - 20 \text{ (TCP Header)} = 65495 \text{ payload}$
- TCP基本协议：滑动窗口协议
- 尽管协议简单、但要解决许多问题



The TCP Segment Header



TCP Header.

The TCP Segment Header (2)

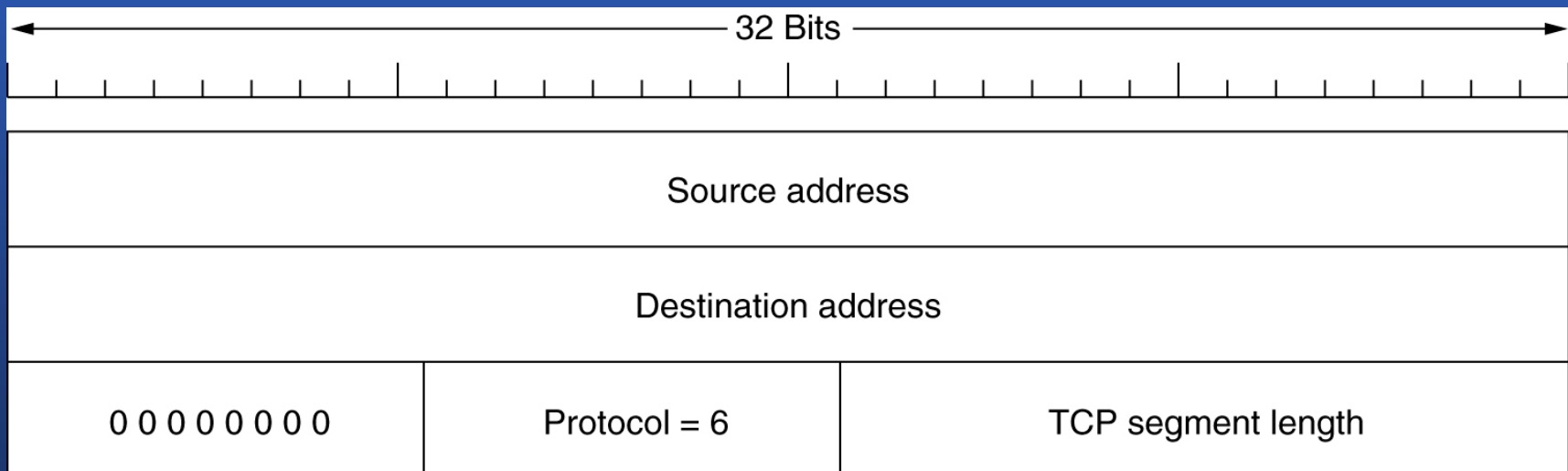
- Source port / Destination port
- Sequence number
- Acknowledgment number
 - The next byte expected, not the last byte correctly received
- Header length
 - How many 32-bit words
- URG =1: Urgent pointer is in use
- ACK =1: Acknowledgment number is valid
- PSH =1: deliver the data to the application layer upon arrival
- RST =1: to reset a connection
- SYN =1: to establish a connection
 - **SYN=1, ACK=0: CONNECTION REQUEST**
 - **SYN=1, ACK=1: CONNECTION ACCEPTED**
- FIN =1: to release a connection



The TCP Segment Header (3)

- Checksum

- Include: Header, Data and Pseudoheader



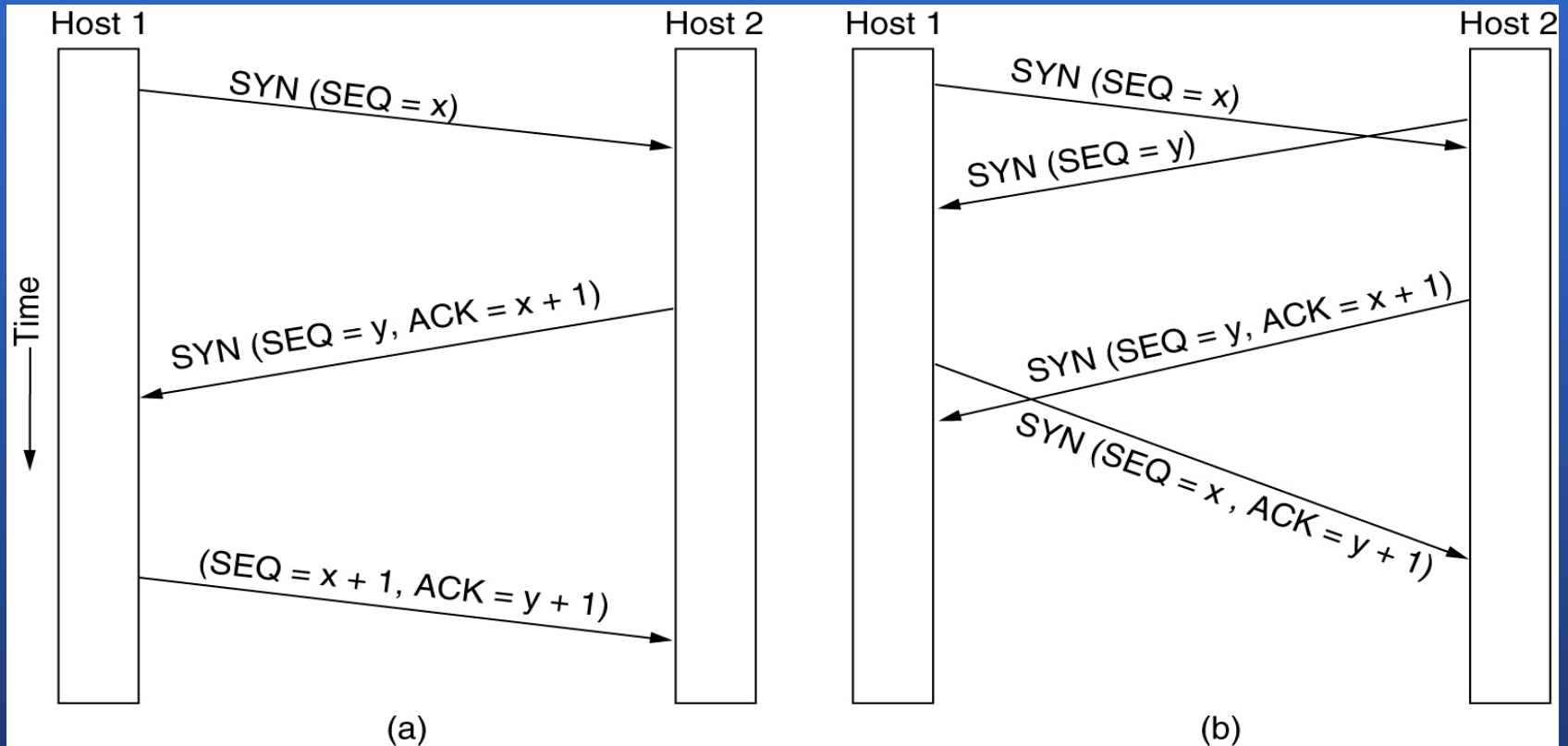
The pseudoheader included in the TCP checksum.

The TCP Segment Header (4)

- Option
 - Maximum TCP payload willing to accept
 - Default 536 bytes → TCP segment: $536 + 20 = 556$
 - Selective repeat transmission (instead of go back n protocol)
 - Option: Windows scale
- Window size



6.5.5 TCP Connection Establishment



(a) TCP connection establishment in the normal case.

(b) Call collision.

TCP Connection Establishment(2)

- 建立连接：采用三次握手的方法
 - 服务器：执行LISTEN和ACCEPT原语
 - 客户端：执行CONNECT原语 (SYN=1, ACK=0)
- 两主机同时想在相同的套接字间建立连接，造成碰撞



6.5.6 TCP Connection Release

- Think of connections as a pair of **simplex connections**
- Each simplex connection is released independently of its sibling
- When the FIN is acknowledged, that direction is shut down for new data
- To avoid two-army problem, timers are used



6.5.7 TCP Connection Management Modeling

State	Description
CLOSED	No connection is active or pending
LISTEN	The server is waiting for an incoming call
SYN RCVD	A connection request has arrived; wait for ACK
SYN SENT	The application has started to open a connection
ESTABLISHED	The normal data transfer state
FIN WAIT 1	The application has said it is finished
FIN WAIT 2	The other side has agreed to release
TIMED WAIT	Wait for all packets to die off
CLOSING	Both sides have tried to close simultaneously
CLOSE WAIT	The other side has initiated a release
LAST ACK	Wait for all packets to die off

The states used in the TCP connection management finite state machine.

TCP Connection Management Modeling (2)

- Event/action

- Event:

- A user-initiated system call(CONNECTION, LISTEN, SEND, CLOSE)
 - A segment arrival(SYN, FIN, ACK, or RST)
 - A case: a timeout of twice the maximum packet lifetime

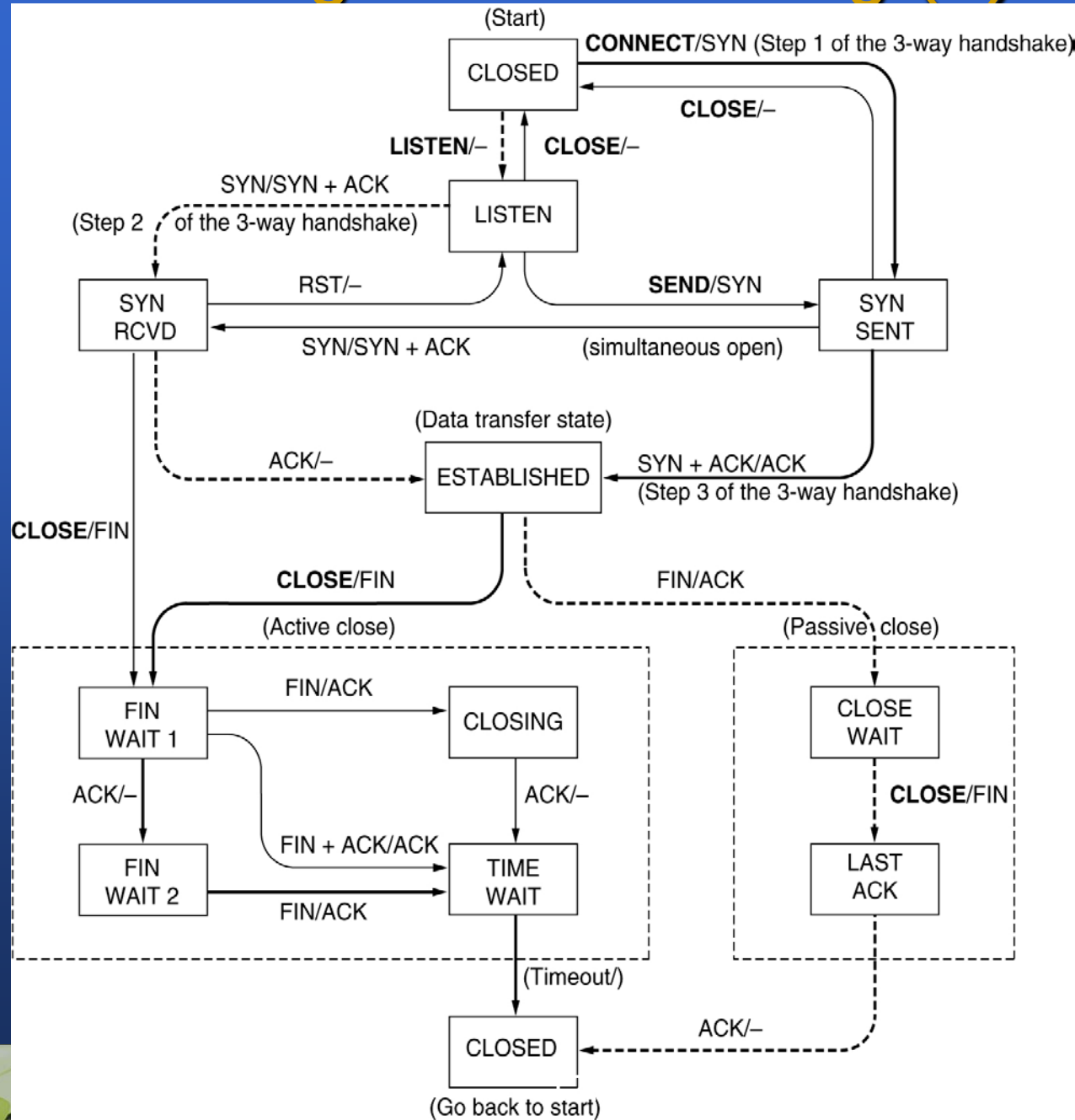
- Action:

- The sending of a control segment (SYN,FIN or RST)
 - Nothing (-)



TCP Connection Management Modeling (3)

- TCP connection management finite state machine.
- The heavy solid line is the normal path for a client.
- The heavy dashed line is the normal path for a server.
- The light lines are unusual events.
- Each **transition** is labeled by the **event** causing it and the **action** resulting from it, separated by a slash. (**event/action**)



TCP Connection Management Modeling (4)

- **Event/Action (Comments)**

- **Event :**

- a user initiated system call (CONNECT, LISTEN, SEND, or CLOSE)
- A segment arrival (SYN, FIN, ACK, or RST)
- A timeout of twice the maximum packet lifetime

- **Action:**

- The sending of a control segment (SYN, FIN, or RST)
- Nothing (indicated by “-”)



TCP Connection Management Modeling (5)

■ 分析:

◆ 沿客户端的路径(粗实线)

1. 客户端应用发出CONNECT请求,本地TCP实体创建一个连接记录,然后发送SYN数据段到服务器端 [CONNECT/SYN] (第一次握手),并标记为SYN SENT状态
2. 当SYN+ACK (SYN=1,ACK=1)数据段到达后,TCP实体发出最后一个ACK数据段[SYN+ACK/ACK] (第三次握手),状态进入ESTABLISHED. 现在可开始数据发送和接收了.
3. 完成数据收发任务后,客户端执行CLOSE原语,使当地TCP实体发送出FIN数据段[CLOSE/FIN],并等待相应的ACK数据段(主动关闭),进入FIN WAIT1状态
4. ACK到达后[ACK/-],转移到FIN WAIT2状态,连接在一个方向上断开了.
5. 另一方面也需要断开连接时,会发出FIN数据段,并得ACK确认 [FIN/ACK]. 现在双方均已断开连接.
6. TCP需要等待一个最大的分组生命期,确保该连接的所有分组全部消失 (TIMED WAIT), 防止出现确认丢失,此时已没有任何Event/Action了[-/-]



TCP Connection Management Modeling (6)

■ 分析:

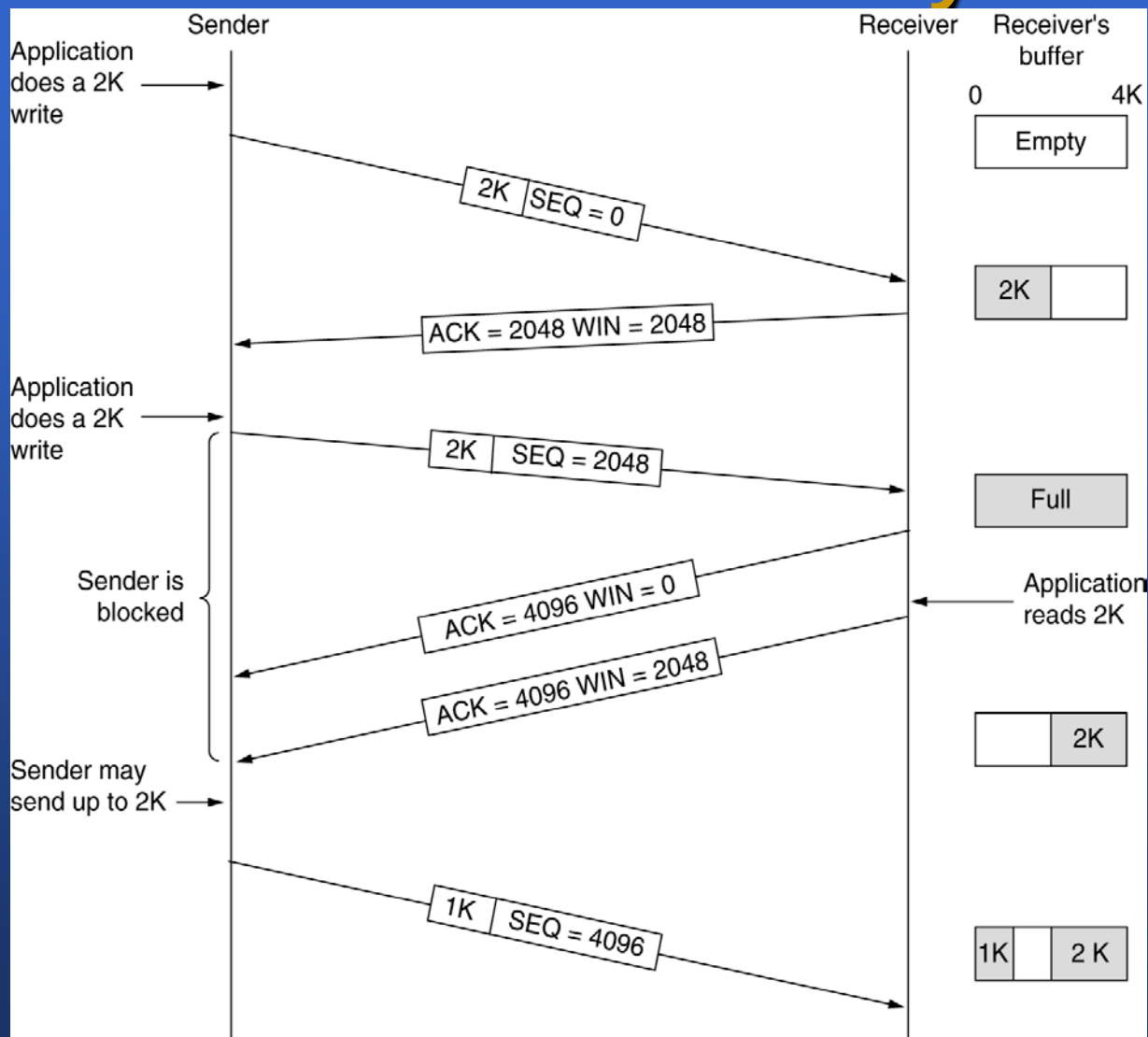
◆ 沿服务端的路径(粗虚线)

1. 服务器执行LISTEN原语,并等待连接请求到来[**LISTEN/-**]. 状态:LISTEN
2. 一个SYN数据段到达后,将被确认 [**SYN/SYN+ACK**](第二次握手), 并且服务器进入SYN REVD状态
3. 当服务器发出的SYN也被确认[**ACK/-**]后,三次握手便完成了, 服务器进入ESTABLISHED状态, 现在可开始数据发送和接收了.
4. 客户端完成数据发送任务后,执行CLOSE原语, 发送一个FIN数据段到达服务器端. (被动关闭).服务器给出响应[**FIN/ACK**]后, 便进入CLOSE WAIT状态.
5. 服务器也执行CLOSE原语, 向客户端发出一个FIN [**CLOSE/FIN**], 进入LAST ACK状态
6. 收到客户端的确认后[**ACK/-**],服务期便释放该连接, 进入CLSOED状态,删除连接记录



6.5.8 TCP Transmission Policy

- 通常，当滑动窗口 $\text{win}=0$ 时，停止发送
 - 但有例外：
 - Urgent data
- 数据传输的最坏情况：Telnet—每个输入字符需要162字节带宽并发送4个数据段



Window management in TCP.

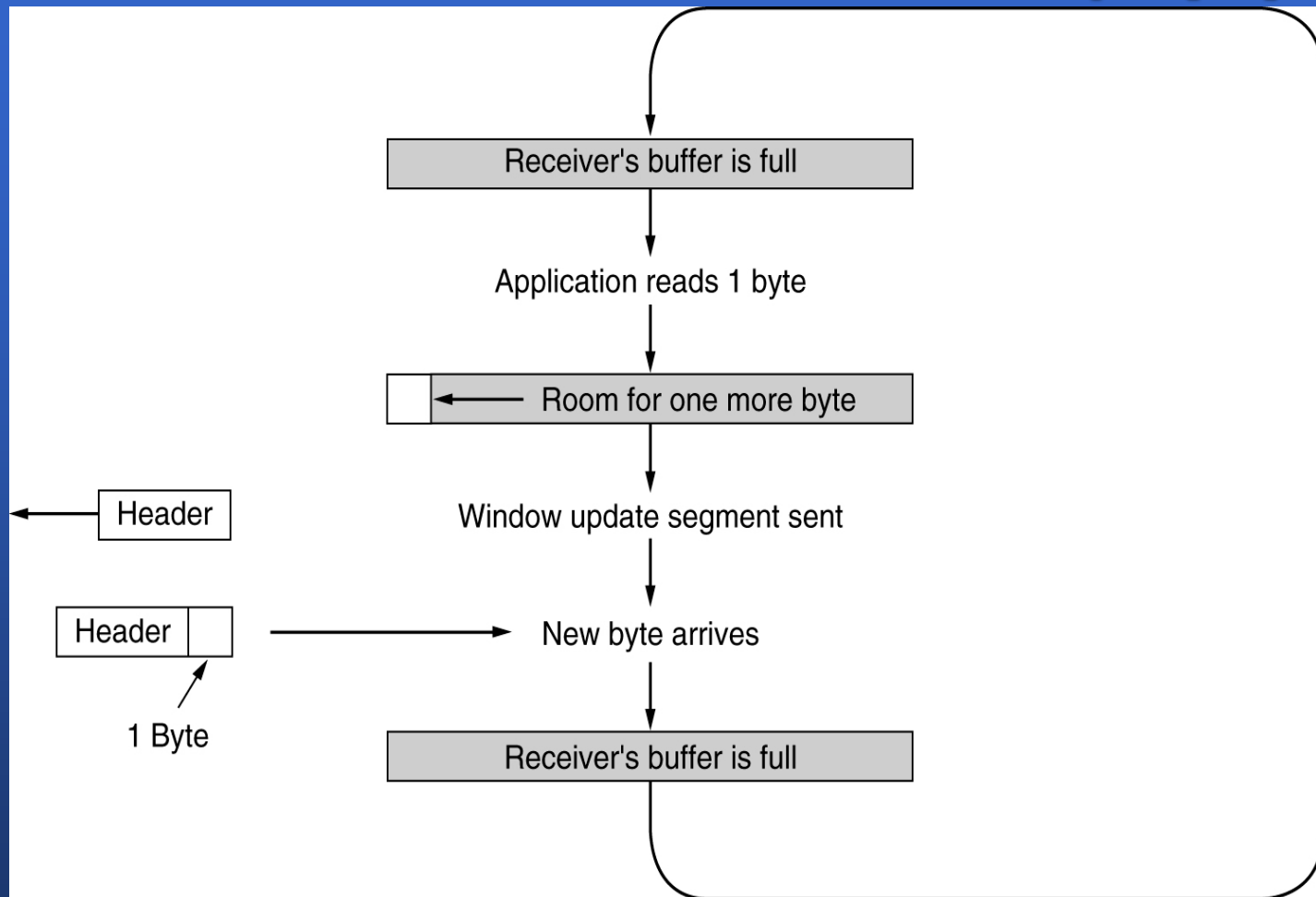


TCP Transmission Policy (2)

- Nagle's algorithm:
 - When data come into the sender one byte at a time, just send the first byte and buffer all the rest until the outstanding byte is acknowledge
 - Try to **solve the problem** caused by the **sender** delivering data to **a byte at a time**



TCP Transmission Policy (3)



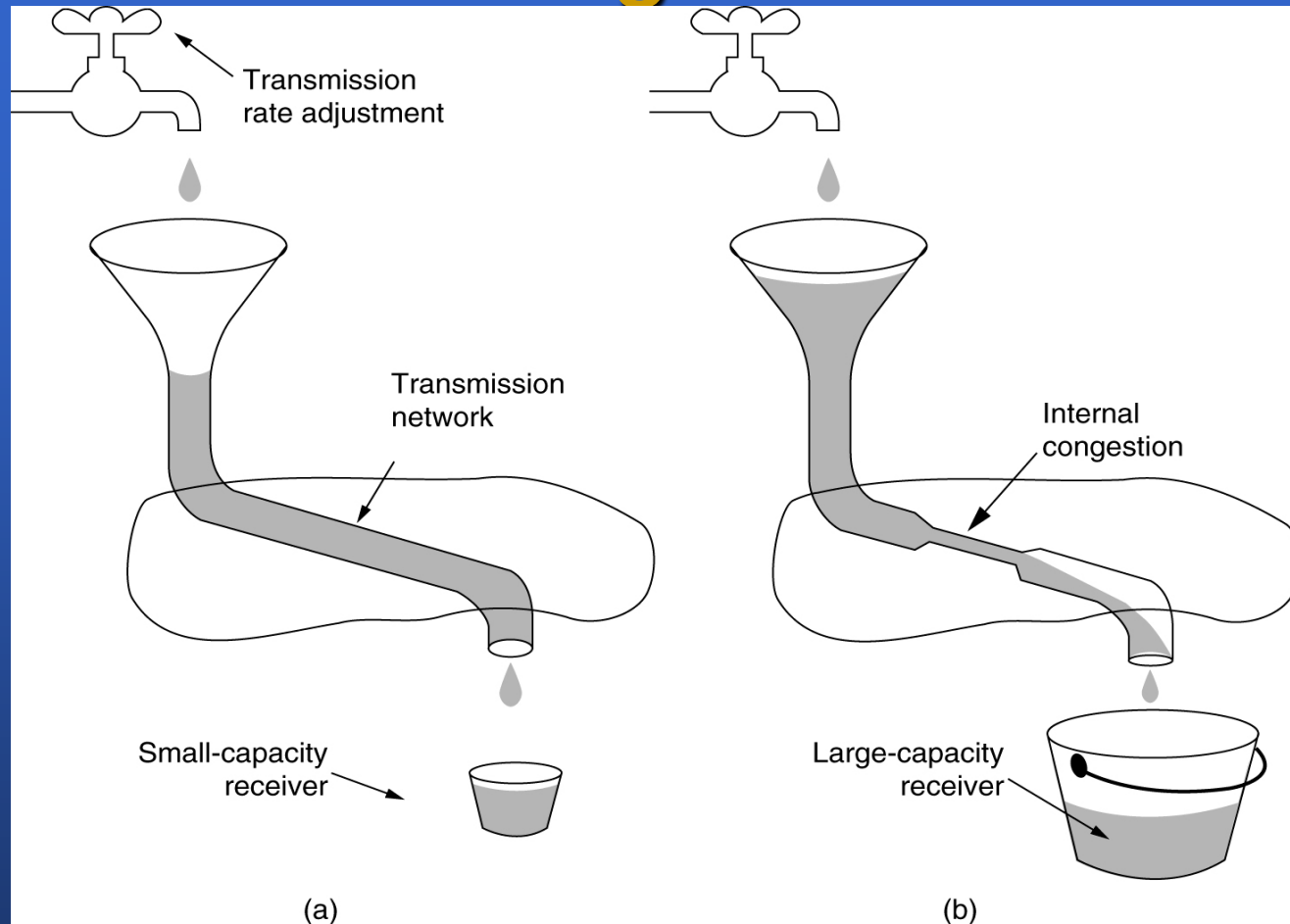
Silly window syndrome(傻瓜窗口症状).

TCP Transmission Policy (4)

- Clark Solution (for silly window syndrome)
 - Forced to wait until it has a decent amount of space available and advertise that to sender
 - To **solve the problem** of the **receiving** application sucking the data up from TCP **a byte at a time**



6.5.9 TCP Congestion Control



(a) A fast network feeding a low capacity receiver.

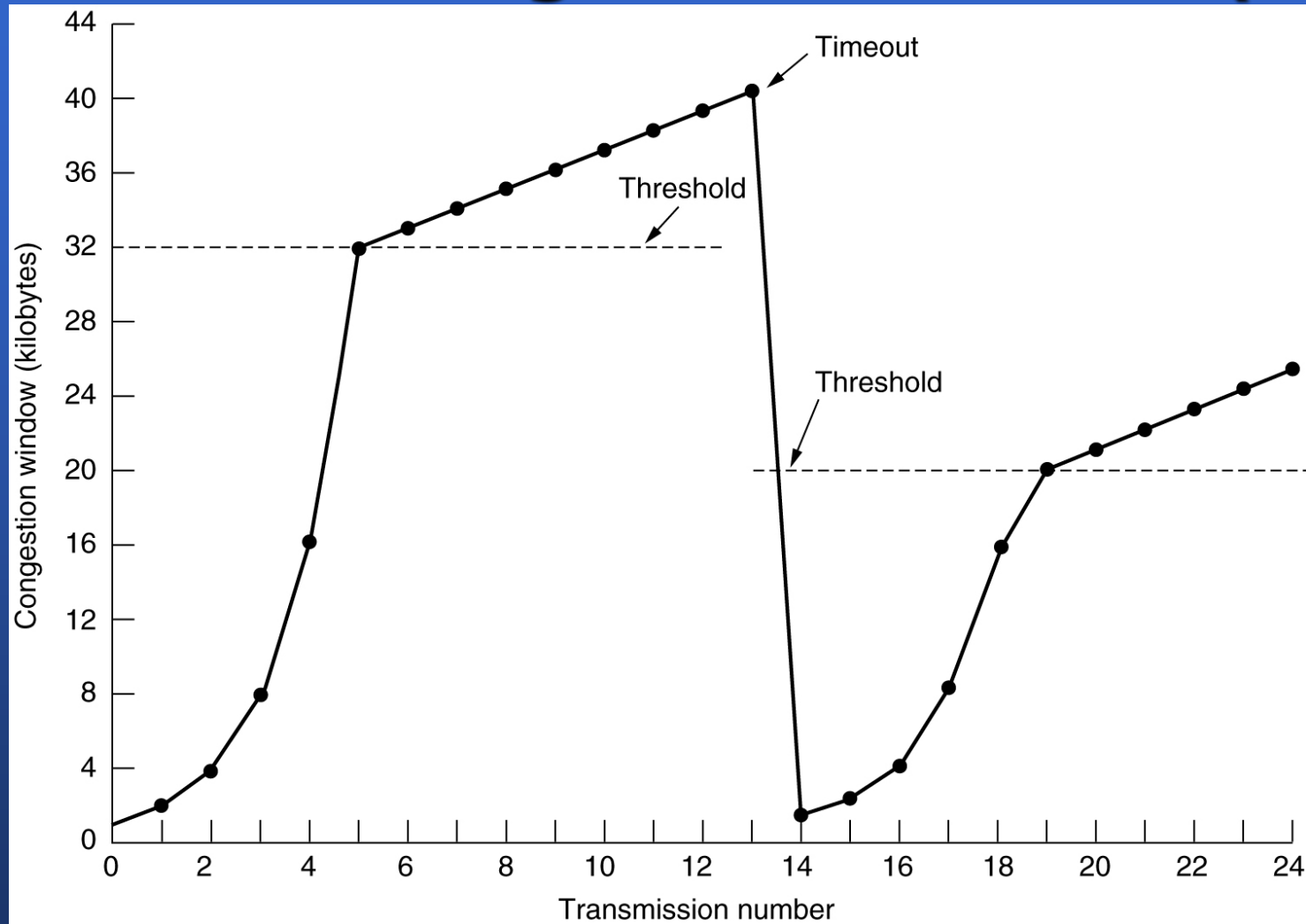
(b) A slow network feeding a high-capacity receiver.

TCP Congestion Control (2)

- 两个潜在的问题：
 - 网络的容量
 - 接收方的容量
- 发送方需要保持的**两个窗口**：
 - **接收方承认的窗口**
 - **拥塞窗口** (Congestion Window)
 - 取上面**两个基本点窗口的最小值**作为可发送字节数
- 拥塞窗口大小的获取：
 - 采用慢速启动 (slow start) 算法



TCP Congestion Control (3)



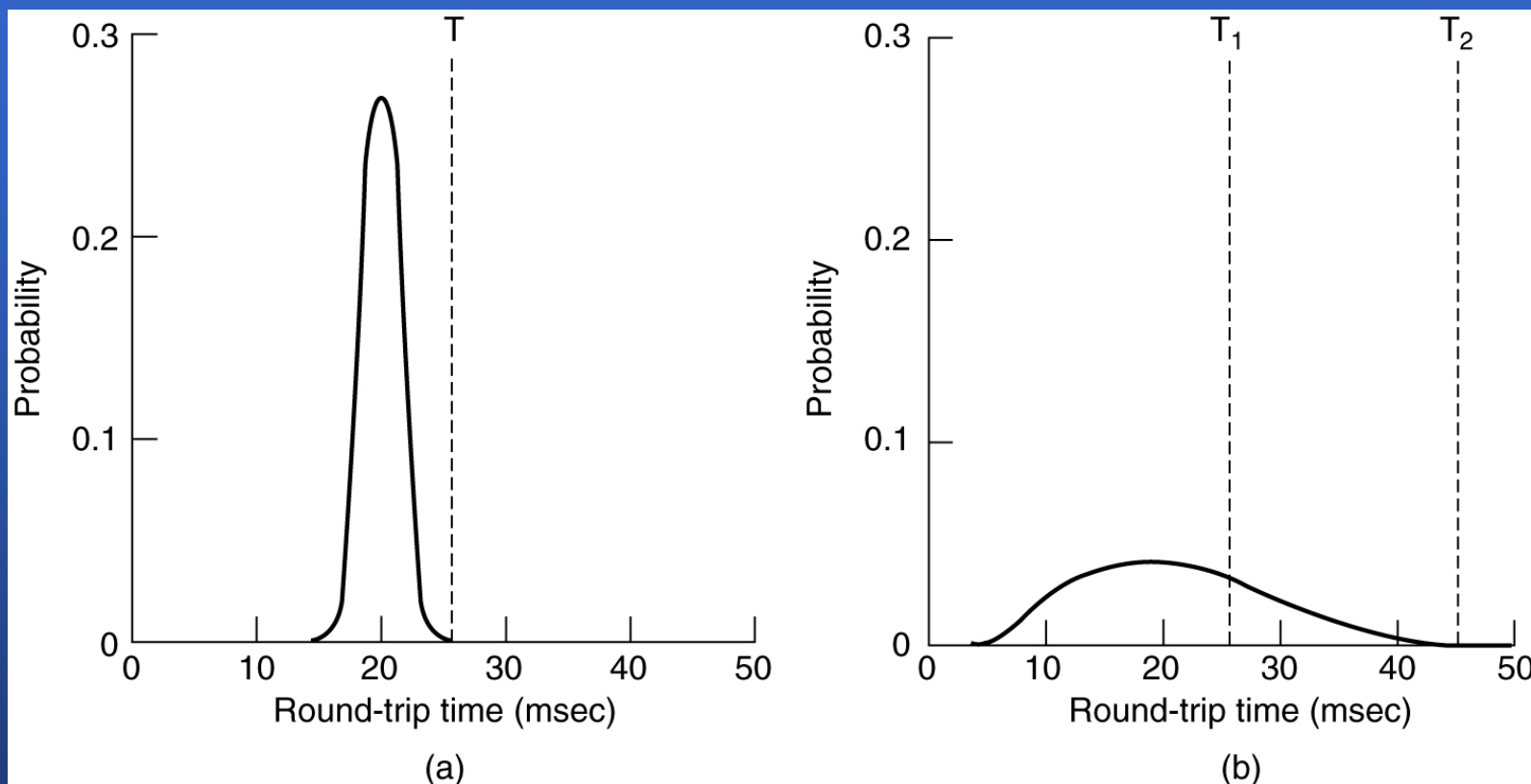
An example of the Internet congestion algorithm.

6.5.10 TCP Timer Management

- 重发定时器(**Retransmission Timer**)
 - 超时时间间隔如何确定?
 - 数据链路层vs TCP中确认到达时间的概率
 - 超时时间间隔太短 (T1)或太长 (T2) 都有问题



TCP Timer Management(2)



(a) Probability density of ACK arrival times in the data link layer.

(b) Probability density of ACK arrival times for TCP.

TCP Timer Management(3)

- 解决办法:

- 通过对网络性能的不断测定, 采用一种不断调整超时时间间隔的动态算法 (Jacobson, 1988)

- 原理:

- $RTT = \alpha RTT + (1 - \alpha) M$

- $D = \alpha D + (1 - \alpha) |RTT - M|$

- 超时值 = $RTT + 4 * D$

- 其中: M—响应 (ACK) 时间, α —smoothing factor RTT—Retransmission Timer

- 简单建议: 已重发的数据段无需修正其RTT, 而是在失败后将超时时间加倍, 直到传输成功。
-- (Karm算法)



TCP Timer Management(4)

- **持续定时器(Persistence Timer)**
 - 防止出现死锁
- **“保活”定时器(Keepalive Timer)**
 - 超时后检测对方是否仍然存在
- **TIMED WAIT定时器**
 - 设定为分组最长生命周期的两倍
 - 确保释放连接创建时申请的所有资源



6.5.11 Wireless TCP and UDP

- 原有的TCP实现都基于有线网，在无线网上可能性能极差
- 主要问题：拥塞控制算法
 - 有线网：分组丢失时，放慢发送速度
 - 无线网：分组丢失时，尽快重发分组

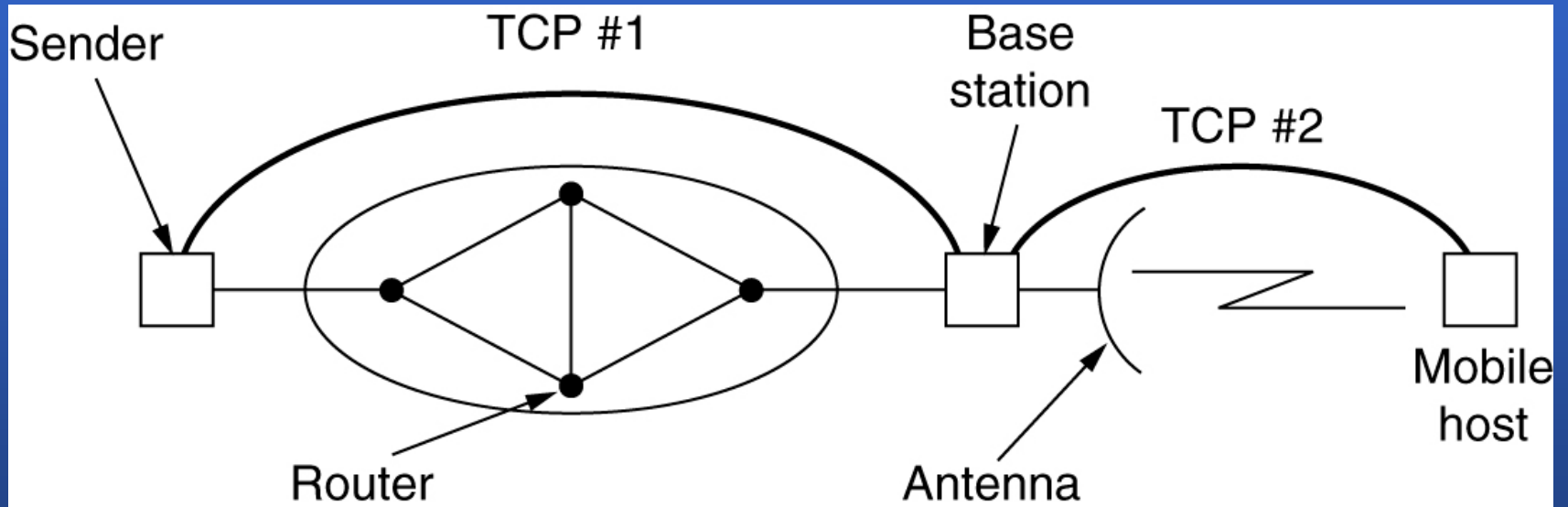


Wireless TCP and UDP (2)

- 解决办法：
 - 采用“间接TCP”
 - 将一个TCP连接分为两个连接-[see fig 6-39]
 - 优点：两个连接均为同质
 - 缺点：不符合TCP的定义
- 对无线网上的UDP同样需要制定一套确保尽量可靠的解决方案

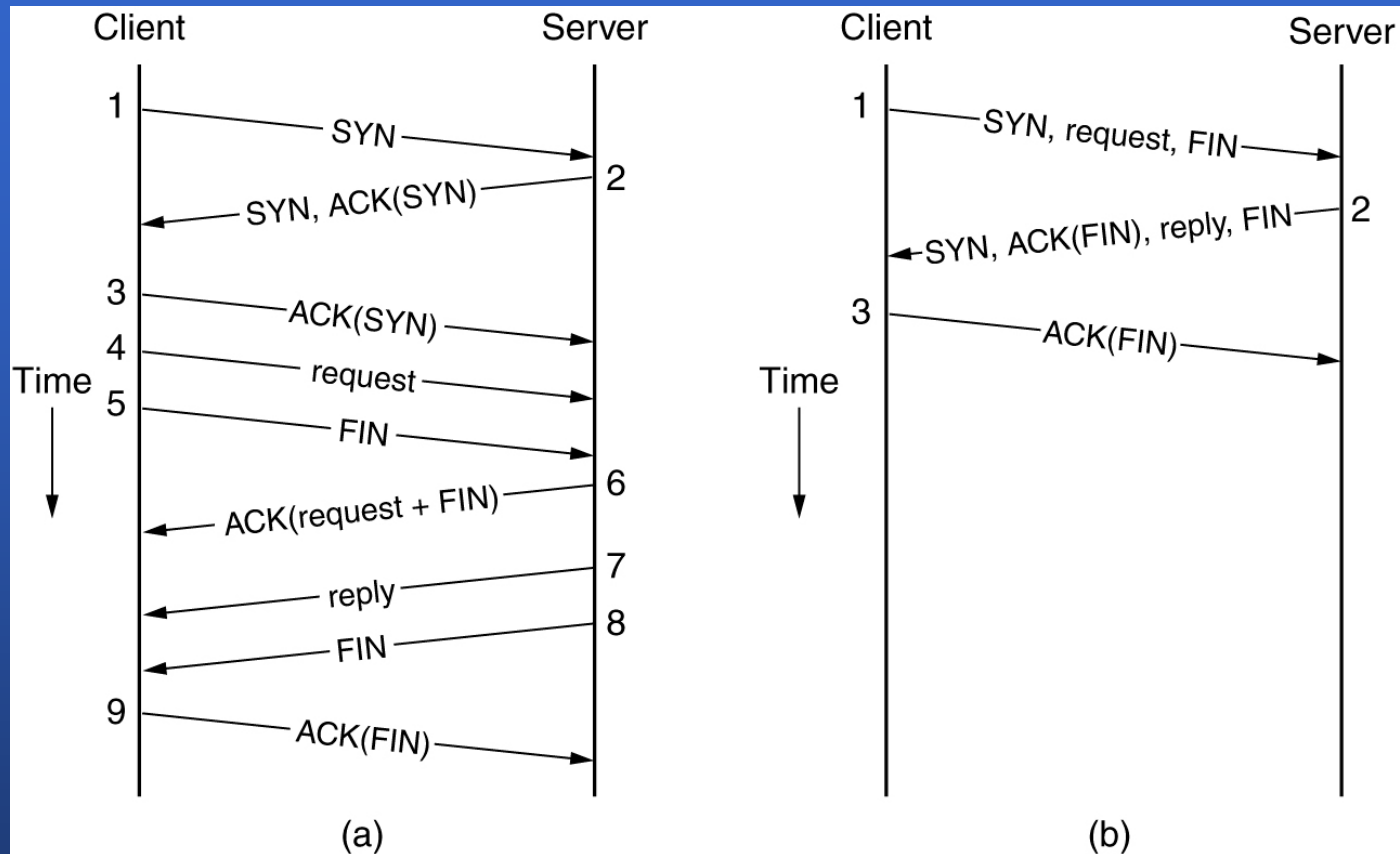


Wireless TCP and UDP (3)



Splitting a TCP connection into two connections.

6.5.12 Transactional TCP



(a) RPC using normal TCP.

(b) RPC using T/TCP.

Transactional TCP(2)

- Client's first packet contains:
 - SYN bit
 - Request
 - FIN
 - “I want to establish a connection, here is the data and I am done.”
- Server:
 - Get the request
 - Look up or compute to form a reply
 - FIN
 - “I acknowledge your FIN, here is the answer, and I am done”



6.6 Performance Issues [*]

- Performance Problems in Computer Networks
- Network Performance Measurement
- System Design for Better Performance
- Fast TPDU Processing
- Protocols for Gigabit Networks



Performance Issues(2)

- 性能问题的复杂性
 - 性能下降却不知道为什么
 - 了解性能更像一门艺术，而不是一门科学
 - 涉及：网路层、传输层等
- 术语
 - “段” (segment) – 传输层
 - “分组” (packet) – 网络层
 - “报文” (message) – 应用层
 - “帧” (frame) – 数据链路层

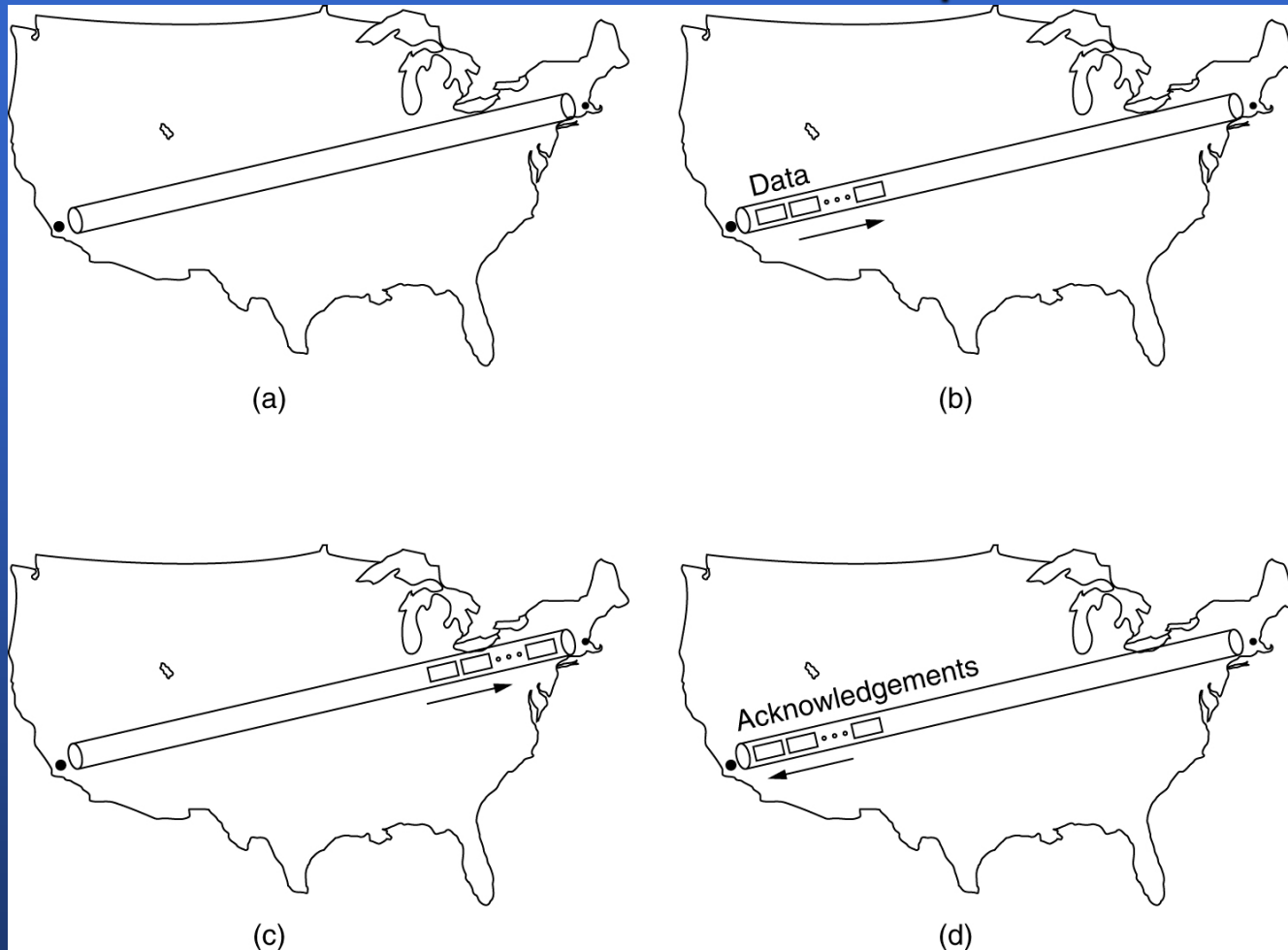


6.6.1 Performance Problems in Computer Networks

- 拥塞
- 资源组织不合理
- 过载，广播风暴(broadcast storm)
 - UDP遭受广播风暴解决方法：主机不对广播方式的UDP错报做出响应
 - 适当地设置超时时限
- 电源故障 – RARP请求
- 千兆级网络上使用原有协议-[see fig 6-41]
 - 带宽延迟乘积(bandwidth-delay product)
 - 结论：若想得到好的性能，接收窗口大小必须大于等于带宽延迟乘积。因为接收方不会马上作出响应



Performance Problems in Computer Networks



The state of transmitting one megabit from San Diego to Boston

(a) At $t = 0$, (b) After $500 \mu\text{sec}$, (c) After 20 msec , (d) after 40 msec .

6.6.2 Network Performance Measurement

The basic loop for improving network performance.

- Measure relevant network parameters, performance.
- Try to understand what is going on.
- Change one parameter.

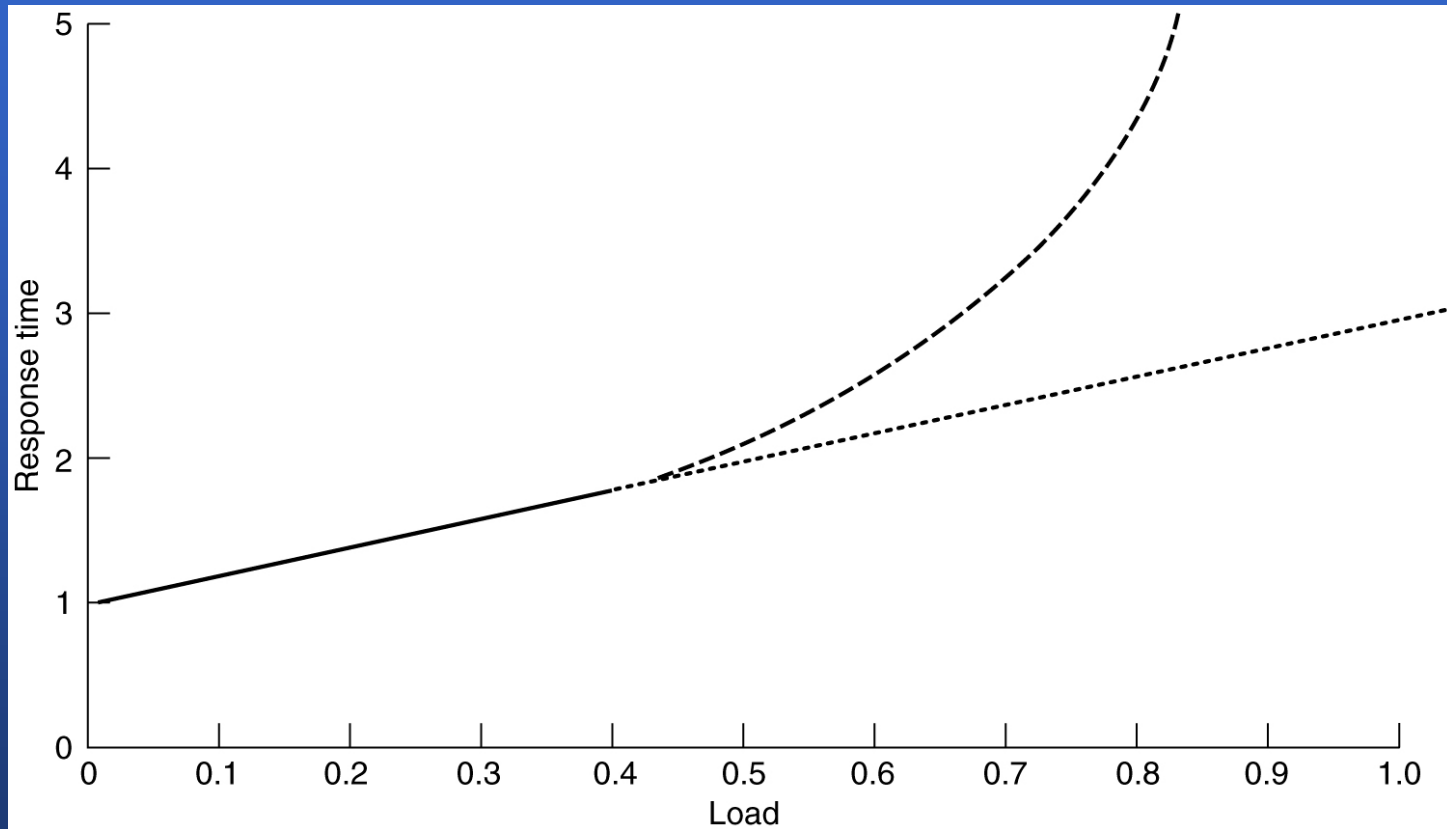


Network Performance Measurement(2)

- 改善性能的基本步骤（循环）：
 - 测定相关的网络参数和性能
 - 尽量弄清发生了什么情况
 - 改变一个参数
- 注意事项：
 - 确保样本空间足够大
 - 确保样本具有代表性
 - 当使用粗粒度时钟时要小心
 - 确保在进行测试期间不会发生不可预知的事情
 - 缓存机制可能会干扰测量结果
 - 明白在测量什么
 - 注意对结果的外推-[see fig 6-42]



Network Performance Measurement(3)



Response as a function of load.

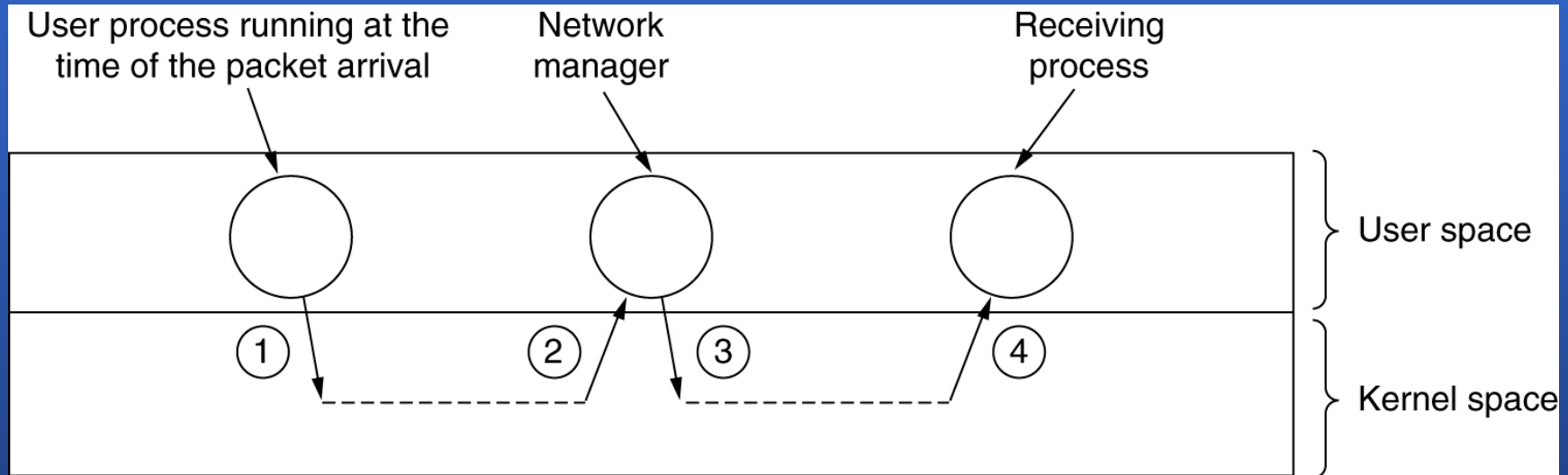
6.6.3 System Design for Better Performance

Rules:

- CPU speed is more important than network speed.
- Reduce packet count to reduce software overhead.
- Minimize context switches.
- Minimize copying.
- You can buy more bandwidth but not lower delay.
- Avoiding congestion is better than recovering from it.
- Avoid timeouts.



System Design for Better Performance (2)



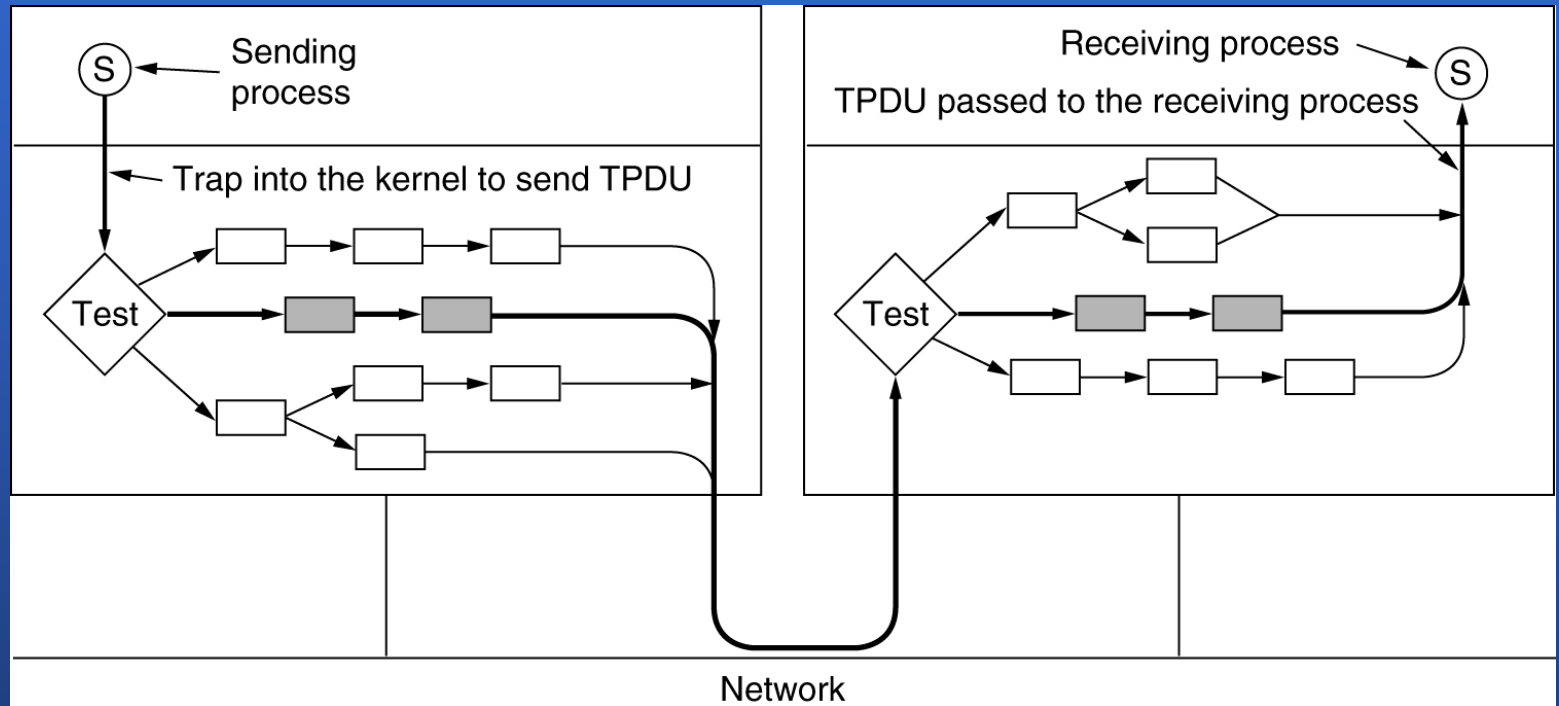
Four context switches to handle one packet with a user-space network manager.

6.6.4 Fast TPDU Processing

- 影响网络速度的主要因素：协议软件
- 对TPDU的处理开销：
 - 对每个TPDU
 - 对每个字节
- 发送/接收进程的处理过程-[see fig 6-44]



Fast TPDU Processing(2)



The fast path from sender to receiver is shown with a heavy line.

The processing steps on this path are shaded.

Fast TPDU Processing (3)

Source port				Destination port			
Sequence number							
Acknowledgement number							
Len		Unused				Window size	
Checksum				Urgent pointer			

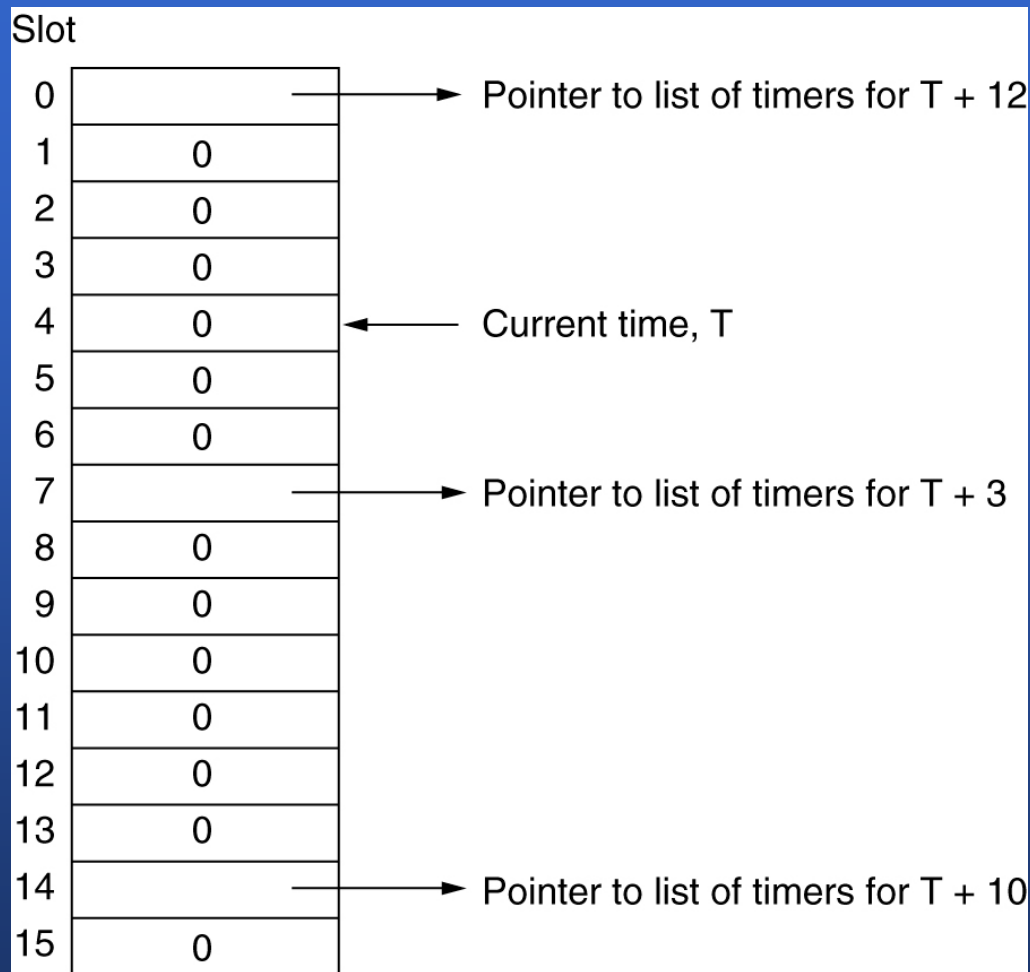
(a)

VER.	IHL	TOS	Total length			
Identification						Fragment offset
TTL		Protocol	Header checksum			
Source address						
Destination address						

(b)

(a) TCP header. (b) IP header. In both cases, the shaded fields are taken from the prototype without change. 连续的数据 TPDU的头几乎相同，可以很快被复制

Fast TPDU Processing (3)



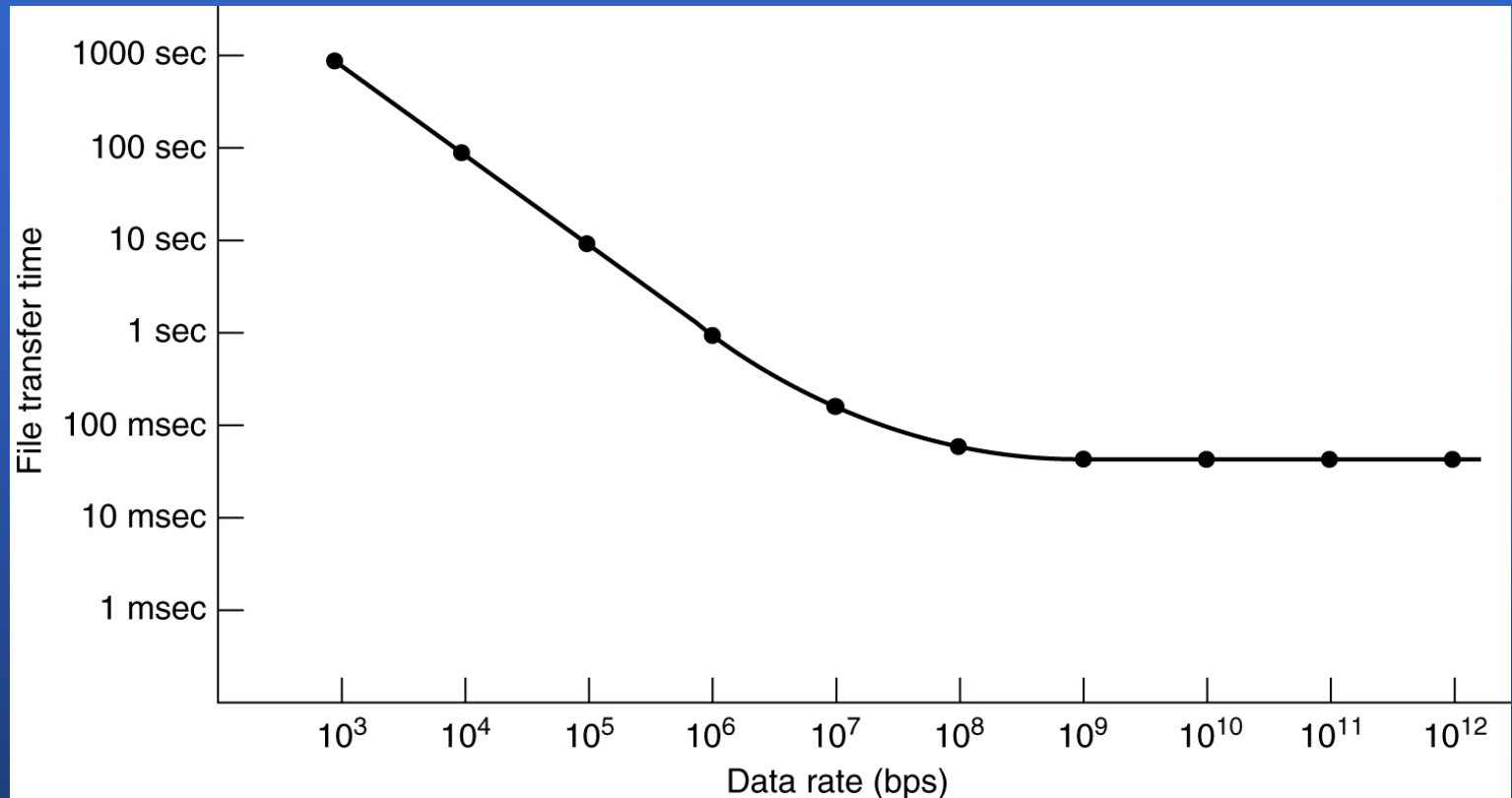
A timing wheel.

6.6.5 Protocols for Gigabit Networks

- 将传统协议用于千兆位网络为产生很多问题
- 问题：
 - 多数协议使用了16位或32位序号
 - 如果用分组编号来取代对字节编号，可以解决问题
 - 通信速度已经比计算机速度还快
 - 协议得到的处理时间减少，因此，协议必须简单
 - 采用回到第N个分组协议使具有大的“带宽-延迟乘积”的线路性能变差
 - 千兆位线路与兆位线路根本不同：长距离线路受延迟时间限制而非带宽限制-[see fig 6-52]
 - 新的大数据量的应用（如多媒体）
 - 慢而均匀的传送速率比快而跳跃的传送速率好



Protocols for Gigabit Networks(2)



Time to transfer and acknowledge a 1-megabit file over a 4000-km line.

Protocols for Gigabit Networks(3)

- 解决方法:

- 为速度设计而不是为带宽而设计
- 应将协议（软件）设计得最小
- 尽量避免反馈（ACK）
- 倾向于使用面向连接的操作
- 分组格式 头部尽量小
- 头和数据应单独进行校验
- 最大数据大小(MTU)应尽可能大
- 随连接请求一起发送正常数据的能力
- 协议软件:
 - 主要思想应集中在操作成功的情况
 - 使复制的时间最短



6.7 Summary

- 传输层的功能、任务
- 在不可靠的网络上提供可靠连接管理
- 因特网的传输协议：TCP和UDP
- 网络性能



Exercises

- 19,23,25,20

- **25, 30, 36**

